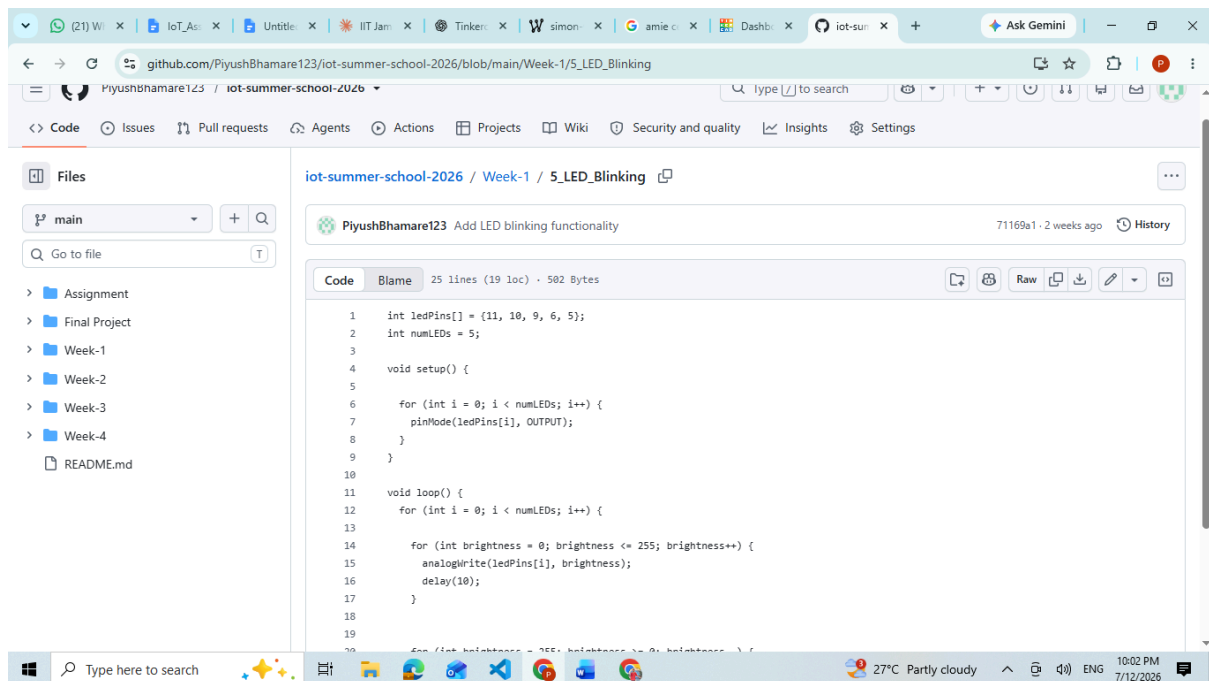


Name :- Bhamare Piyush Girish
Course :- IoT & Drone Building
Assignment Q. 1 - 35

Q1. GitHub Repository Creation

<https://github.com/PiyushBhamare123/iot-summer-school-2026>

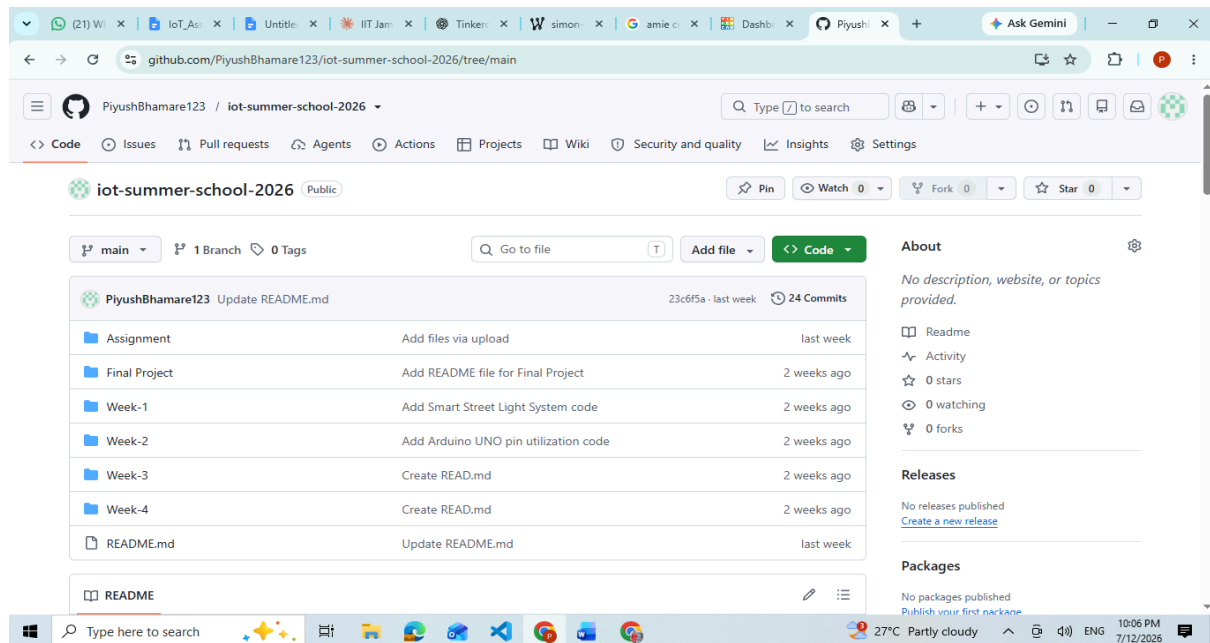
Q2. Git Workflow – Push Your First Code



The screenshot displays a web browser window showing a GitHub repository page. The browser's address bar shows the URL `github.com/PiyushBhamare123/iot-summer-school-2026/blob/main/Week-1/5_LED_Blinking`. The repository name is `iot-summer-school-2026` by user `PiyushBhamare123`. The file being viewed is `5_LED_Blinking` in the `Week-1` directory. The code is written in C++ and implements an LED blinking function. The code is as follows:

```
1  int ledPins[] = {11, 10, 9, 6, 5};
2  int numLEDs = 5;
3
4  void setup() {
5
6      for (int i = 0; i < numLEDs; i++) {
7          pinMode(ledPins[i], OUTPUT);
8      }
9  }
10
11 void loop() {
12     for (int i = 0; i < numLEDs; i++) {
13
14         for (int brightness = 0; brightness <= 255; brightness++) {
15             analogWrite(ledPins[i], brightness);
16             delay(10);
17         }
18     }
19
20     for (int brightness = 255; brightness >= 0; brightness--) {
```

Q3. Branching Strategy



git checkout main

git pull origin main

git checkout -b dev

git checkout -b sensor-module

git checkout dev

git checkout -b wireless-module

git checkout dev

touch Week-1/notes.md

code Week-1/notes.md

git status

git add Week-1/notes.md

git commit -m "docs: add branch plan notes"

git checkout main

git pull origin main

git merge dev --no-ff -m "merge: integrate dev branch notes into main"

git push origin main

```
git push -u origin dev
git push -u origin sensor-module
git push -u origin wireless-module
```

```
git branch -a
```

```
git log --all --graph --oneline --decorate
```

Q4. Versioned Code – Multiple Commits

The screenshot shows a GitHub repository page for 'PiyushBhamare123 / iot-summer-school-2026'. The left sidebar shows the file structure with 'Week-1' selected. The main content area displays the commit history for 'Week-1'.

Name	Last commit message	Last commit date
..		
4*4_By_NestedLoop	Create 4*4_By_NestedLoop	2 weeks ago
4*4_LED_Matrix	Add 4x4 LED Matrix control code	2 weeks ago
5_LED_Blinking	Add LED blinking functionality	2 weeks ago
README.md	Add Week 1 header to README	2 weeks ago
Smart_Street_Light_System	Add Smart Street Light System code	2 weeks ago
Traffic_Light	Add Traffic Light simulation code	2 weeks ago

Q5. README Documentation

The screenshot shows the README.md file content for the 'iot-summer-school-2026' repository. The file is titled 'Summer School on IoT & Embedded Systems – IIT Jammu'.

This repository contains the assignments, practical experiments, simulations, source code, and project work completed during my Summer School at IIT Jammu.

The program provided hands-on experience in Internet of Things (IoT), Embedded Systems, ESP32 programming, sensor interfacing, communication protocols, circuit simulation, and IoT-based project development.

About Me

I am Piyush Bhamare, a B.Tech student specializing in Robotics and Artificial Intelligence, with interests in Robotics, IoT, Embedded Systems, Artificial Intelligence, and Industrial Automation.

Technologies & Tools

<https://github.com/PiyushBhamare123/iot-summer-school-2026/blob/main/README.md>

Q6. Issue Tracking & Milestones

Issues

Assigned to me

Created by me

Mentioned

Recent activity

Views

Projects

Milestones

Labels

Feedback

Preview

Collapse sidebar

is:issue state:open

New issue

Open 3 Closed 1

Author Labels Projects Milestones Assignees Newest

Enhancement: Log data to Serial Monitor in CSV format enhancement

#3 · PiyushBhamare123 opened 4m ago

Feature: Add RGB LED support enhancement

#2 · PiyushBhamare123 opened 4m ago

Bug : Blink interval not changing when potentiometer is at maximum bug

#1 · PiyushBhamare123 opened 4m ago

Give feedback

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

27°C Clear

10:28 PM

7/12/2026

Issues

Assigned to me

Created by me

Mentioned

Recent activity

Views

Projects

Milestones

Labels

Feedback

Preview

Collapse sidebar

Milestones

New milestone

Open 1 Closed 0

Sort

Week 1 Completion

Completion milestone for Git, GitHub, version control, issue tracking, and documentation activities of Week 1.

100% complete 0 open 3 closed

No due date · 3/3 issues closed

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Puppy playfully nibbl...

10:32 PM

7/12/2026

Q7. Collaborative Fork & Pull Request

The screenshot shows the GitHub interface for the repository `PiyushBhamare123 / iot-summer-school-2026`. The 'Pull requests' tab is selected, showing a message: 'There aren't any open pull requests. You could search all of GitHub or try an advanced search.' Above this message, there is a notification banner: 'Label issues and pull requests for new contributors. Now, GitHub will help potential first-time contributors discover issues labeled with good first issue.' The interface includes filters (0 Open, 0 Closed), a search bar (is:pr is:open), and buttons for 'Labels' (9), 'Milestones' (1), and 'New pull request'. The Windows taskbar at the bottom shows the time as 10:36 PM on 7/12/2026.

Q8. CI/CD with GitHub Actions

The screenshot shows the GitHub interface for the repository `PiyushBhamare123 / iot-summer-school-2026`, specifically the `.github/workflows/blank.yml` file. The 'Code' tab is selected, displaying the workflow file content. The workflow is a basic CI setup with the following structure:

```
1 # This is a basic workflow to help you get started with Actions
2
3 name: CI
4
5 # Controls when the workflow will run
6 on:
7   # Triggers the workflow on push or pull request events but only for the "main" branch
8   push:
9     branches: [ "main" ]
10  pull_request:
11    branches: [ "main" ]
12
13 # Allows you to run this workflow manually from the Actions tab
14 workflow_dispatch:
15
16 # A workflow run is made up of one or more jobs that can run sequentially or in parallel
17 jobs:
18   # This workflow contains a single job called "build"
```

The left sidebar shows the file explorer with the following structure:

- `.github/workflows`
- `Assignment`
- `Final Project`
- `Week-1`
- `Week-2`
- `Week-3`
- `Week-4`
- `README.md`

The Windows taskbar at the bottom shows the time as 10:40 PM on 7/12/2026.

Q9. Explain the difference between git pull, git fetch, and git clone. When would you use each? (2 marks)

Git is a distributed version control system that allows developers to maintain a local copy of a project and synchronize it with a remote repository such as GitHub. The commands git clone, git fetch, and git pull are used to obtain data from a remote repository, but each command performs a different operation.

1. git clone

The git clone command is used to create a new local copy of an existing remote repository.

When git clone is executed, Git downloads the complete repository, including the project files, commit history, branches, and tags. It also automatically creates a connection to the remote repository named origin.

Syntax:

```
git clone <repository-url>
```

Example:

```
git clone https://github.com/PiyushBhamare123/iot-summer-school-2026.git
```

When to use:

git clone is used when we want to download a repository to our computer for the first time and start working on the project locally.

2. git fetch

The git fetch command is used to download the latest commits, branches, and tags from a remote repository without changing the current working files.

It updates the information about the remote repository in the local Git database. However, it does not automatically merge the downloaded changes into the current local branch.

Syntax:

```
git fetch <remote-name>
```

Example:

```
git fetch origin
```

When to use:

git fetch is used when we want to check and inspect the latest changes available in the remote repository before deciding whether to merge them into the local branch.

3. git pull

The git pull command is used to download the latest changes from a remote repository and immediately integrate them into the currently checked-out local branch.

By default, git pull performs two operations:

git pull = git fetch + git merge

First, Git downloads the latest changes from the remote repository, and then it merges those changes into the current local branch.

Syntax:

git pull <remote-name> <branch-name>

Example:

git pull origin main

When to use:

git pull is used when we want to update the current local branch with the latest changes from the remote repository and are ready to integrate those changes immediately.

COMPARISON BETWEEN git clone, git fetch, AND git pull

git clone:

Purpose: Creates a new local copy of a remote repository.

Effect on working files: Creates a new working directory containing the repository files.

Automatic merging: Not applicable because the repository is being downloaded for the first time.

Usage: Used when starting work on an existing remote repository.

git fetch:

Purpose: Downloads the latest commits, branches, and tags from the remote repository.

Effect on working files: Does not change the current working files.

Automatic merging: No.

Usage: Used when we want to inspect remote changes before integrating them.

git pull:

Purpose: Downloads and integrates remote changes into the current local branch.

Effect on working files: Updates the working files after integration.

Automatic merging: Yes, by default.

Usage: Used when we want to update the current local branch immediately.

CONCLUSION

git clone is used to create a local copy of a remote repository for the first time.

git fetch downloads the latest changes from the remote repository without modifying the current working branch.

git pull downloads the latest remote changes and integrates them into the current local branch.

The simple rule to remember is:

Clone to start, fetch to inspect, and pull to update and merge.

Q10. What is a .gitignore file? Write a .gitignore for an Arduino project that excludes compiled output files (.hex, .elf), OS-specific files (.DS_Store, Thumbs.db), and IDE config folders (.vscode/, build/). (2 marks)

A .gitignore file is a special file used in a Git repository to specify files and directories that Git should intentionally ignore and not track.

During software, Arduino, embedded systems, and IoT project development, many unnecessary files are automatically generated by compilers, operating systems, development environments, and build tools. IoT projects may also contain sensitive files that store Wi-Fi passwords, API keys, authentication tokens, and other credentials.

The .gitignore file helps prevent these unnecessary and sensitive files from being accidentally added to the Git repository and uploaded to GitHub.

IMPORTANCE OF THE .gitignore FILE

1. Prevents Compiled Files from Being Committed

Arduino and embedded system projects may generate compiled output files such as:

*.hex

*.elf

*.o

*.bin

These files are automatically generated when the source code is compiled. Since they can be regenerated from the source code, they normally do not need to be stored in the Git repository.

2. Prevents Build Directories from Being Committed

Development tools and compilers may create build directories containing temporary and compiled files.

Example:

build/

These files are automatically generated and can make the repository unnecessarily large. Therefore, build directories should normally be ignored.

3. Prevents Operating-System-Specific Files

Operating systems automatically generate some files that are unrelated to the actual project.

Examples:

.DS_Store

Thumbs.db

.DS_Store is commonly generated by macOS, while Thumbs.db is commonly generated by Windows.

These files should not normally be uploaded to the GitHub repository.

4. Prevents IDE Configuration Files

Integrated Development Environments (IDEs) and code editors may create configuration files and directories containing user-specific settings.

Examples:

.vscode/

.vs/

*.iml

These files may contain settings specific to a developer's computer and are usually unnecessary for other contributors.

5. Protects Sensitive Information

IoT applications frequently require sensitive information such as:

Wi-Fi network names (SSIDs)

Wi-Fi passwords

API keys

Authentication tokens

MQTT usernames and passwords

Cloud service credentials

Files such as config.h and secrets.h may contain this confidential information.

These files should be added to the .gitignore file to prevent sensitive information from being accidentally uploaded to a public GitHub repository.

EXAMPLE OF A .gitignore FILE

Compiled Arduino output files

*.hex

*.elf

*.o

*.bin

Build directory

build/

Operating system files

.DS_Store

Thumbs.db

IDE configuration files

.vscode/

.vs/

*.iml

Sensitive information

config.h

secrets.h

IMPORTANT LIMITATION OF .gitignore

The .gitignore file normally affects files that are not already being tracked by Git.

If a file has already been committed to the Git repository, simply adding its name to .gitignore does not automatically stop Git from tracking the file.

For example, if secrets.h has already been committed, the following command can be used to remove the file from Git tracking:

```
git rm --cached secrets.h
```

The change should then be committed and pushed to the remote repository.

If a real password, API key, or authentication token has already been uploaded to a public GitHub repository, removing the file is not sufficient because the sensitive information may still exist in the Git commit history. In such cases, the exposed password, key, or token should also be changed or revoked.

ADVANTAGES OF USING .gitignore

The main advantages of using a .gitignore file are:

1. It keeps the Git repository clean and organized.
2. It prevents unnecessary compiled and temporary files from being committed.
3. It reduces the size of the repository.
4. It prevents operating-system-specific files from being uploaded.
5. It prevents developer-specific IDE configuration files from being shared unnecessarily.
6. It helps protect sensitive information such as Wi-Fi passwords and API keys.
7. It improves collaboration between developers.
8. It makes the repository easier to maintain.

CONCLUSION

The .gitignore file tells Git which files and directories should not be tracked.

It is particularly important in Arduino, embedded systems, and IoT projects because these projects may contain compiled output files, build directories, operating-system files, IDE configuration files, temporary files, Wi-Fi credentials, API keys, and other sensitive information.

Therefore, using a properly configured .gitignore file helps keep the Git repository clean, organized, secure, and easy to maintain.

Q11. IoT Architecture

The Internet of Things (IoT) is a system of physical devices, sensors, actuators, communication networks, data-processing platforms, and applications that work together to collect, transmit, process, and use real-world data.

A commonly used IoT architecture consists of four layers:

1. Perception Layer
2. Network Layer
3. Processing Layer
4. Application Layer

1. PERCEPTION LAYER

The Perception Layer is the lowest layer of the IoT architecture. It directly interacts with the physical environment.

The main function of this layer is to collect real-world information using sensors and perform physical actions using actuators.

Examples of devices used at the Perception Layer include DHT11 temperature and humidity sensors, HC-SR04 ultrasonic sensors, LDR sensors, PIR motion sensors, RFID readers, cameras, motors, relays, and LEDs.

Example Device: DHT11 Temperature and Humidity Sensor.

2. NETWORK LAYER

The Network Layer transfers sensor data from IoT devices to gateways, servers, and cloud platforms. It also transfers control commands from applications to IoT devices.

Communication technologies used at this layer include Wi-Fi, Bluetooth, BLE, Zigbee, LoRaWAN, Ethernet, cellular networks, and NB-IoT.

Example Technology: Wi-Fi.

3. PROCESSING LAYER

The Processing Layer stores, processes, filters, analyzes, and manages the data received from IoT devices.

This layer may contain IoT gateways, edge computing devices, cloud computing platforms, databases, analytics systems, and message brokers.

The Processing Layer can perform data filtering, data aggregation, protocol conversion, rule-based decision making, cloud storage, and data analytics.

Example Technology: Cloud Computing Platform.

4. APPLICATION LAYER

The Application Layer provides services and interfaces to end users.

Users can monitor sensor readings, receive alerts, control devices, view reports, and analyze IoT data through mobile applications, websites, dashboards, and other software systems.

Examples include smart homes, smart cities, industrial automation, healthcare monitoring, smart agriculture, and intelligent transportation systems.

Example Application: Smart Home Monitoring Dashboard.

FOUR-LAYER IoT ARCHITECTURE DIAGRAM

Physical Environment

↓

PERCEPTION LAYER

Sensors: DHT11, LDR, PIR, HC-SR04

Actuators: Motors, LEDs, Relays

↓

NETWORK LAYER

Wi-Fi, Bluetooth, Zigbee, LoRaWAN

↓

PROCESSING LAYER

IoT Gateway, Edge Computing, Cloud Server, Database

↓

APPLICATION LAYER

Mobile Application, Web Dashboard, Alerts, Reports

↓

End User

CONCLUSION

The four-layer IoT architecture provides a structured model for understanding the operation of IoT systems.

The Perception Layer collects physical data.

The Network Layer transfers the collected information.

The Processing Layer stores and analyzes the information.

The Application Layer provides useful IoT services to users.

Q12. Microcontroller vs Microprocessor

A microcontroller and a microprocessor are programmable electronic devices used to perform computational and control operations. However, their internal architecture, applications, power consumption, cost, and peripheral requirements are different.

A microcontroller is a complete small computer system integrated into a single chip.

It normally contains a CPU, RAM, Flash memory, GPIO pins, timers, ADC, PWM modules, and communication interfaces.

Examples include ATmega328P, ESP32, STM32, and PIC microcontrollers.

A microprocessor mainly contains a Central Processing Unit. External RAM, storage, input/output controllers, and other peripheral devices are generally required to build a complete system.

Examples include Intel Core processors, AMD Ryzen processors, and ARM application processors.

COMPARISON

Microcontroller:

1. Contains CPU, memory, and peripherals on a single chip.
2. Designed mainly for dedicated embedded applications.
3. Consumes relatively less electrical power.
4. Generally has lower processing capability than a general-purpose microprocessor.
5. Lower system cost.
6. Suitable for real-time control applications.
7. Commonly used in IoT devices, robots, washing machines, automobiles, and industrial control systems.
8. Examples include Arduino UNO's ATmega328P, ESP32, STM32, and PIC.

Microprocessor:

1. Mainly contains the CPU and requires external components to build a complete computer system.
2. Designed for general-purpose and computation-intensive applications.

3. Consumes relatively more electrical power.
4. Generally provides higher computational capability.
5. Higher overall system cost and complexity.
6. Suitable for operating systems and complex software applications.
7. Commonly used in desktop computers, laptops, servers, and high-performance embedded systems.
8. Examples include Intel Core, AMD Ryzen, and application-class ARM processors.

CONCLUSION

Microcontrollers are suitable for embedded systems and IoT applications that require low cost, low power consumption, and direct interaction with sensors and actuators.

Microprocessors are suitable for systems that require high processing capability, large memory, complex operating systems, and general-purpose computation.

Q13. Arduino Pin Types

Arduino UNO provides different types of input, output, and communication pins.

1. DIGITAL INPUT

Digital input pins read two logical states: HIGH or LOW.

The `digitalRead()` function is used to read digital input.

Example:

```
digitalRead(2);
```

IoT Use Case:

A PIR motion sensor can be connected to a digital input pin to detect human movement in a smart security system.

2. DIGITAL OUTPUT

Digital output pins generate HIGH or LOW output signals.

The `digitalWrite()` function is used to control digital outputs.

Example:

```
digitalWrite(13, HIGH);
```

IoT Use Case:

A relay connected to a digital output can switch a water pump in a smart irrigation system.

3. ANALOG INPUT

Arduino UNO contains six analog input pins, A0 to A5.

The ATmega328P uses a 10-bit ADC, so `analogRead()` typically returns values from 0 to 1023.

Example:

```
analogRead(A0);
```

IoT Use Case:

An LDR can be connected to an analog input to measure light intensity in a smart street-light system.

4. PWM OUTPUT

Arduino UNO contains six PWM-capable pins:

D3, D5, D6, D9, D10, and D11.

The `analogWrite()` function is used to generate PWM output.

Example:

```
analogWrite(9, 128);
```

IoT Use Case:

PWM can control the brightness of an LED or the speed of a DC motor.

5. I2C PINS

Arduino UNO uses:

A4 — SDA

A5 — SCL

SDA transfers data.

SCL carries the clock signal.

IoT Use Case:

An I2C OLED display can display environmental sensor readings.

6. SPI PINS

Arduino UNO uses:

D10 — SS

D11 — MOSI

D12 — MISO

D13 — SCK

IoT Use Case:

An SD card module can store sensor data locally using SPI communication.

CONCLUSION

Arduino UNO supports digital input, digital output, analog input, PWM output, I2C, and SPI communication. These interfaces allow Arduino to interact with sensors, actuators, displays, memory devices, and communication modules.

Q14. Traffic Light Controller

Main Code -

```
const int RED_LED = 10;
const int YELLOW_LED = 9;
const int GREEN_LED = 8;
const int BUTTON_PIN = 2;

void setup() {
  pinMode(RED_LED, OUTPUT);
  pinMode(YELLOW_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);

  pinMode(BUTTON_PIN, INPUT_PULLUP);

  Serial.begin(9600);
  Serial.println("Traffic Light Controller Started");
}

void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) {
    pedestrianSequence();
    return;
  }

  Serial.println("GREEN LIGHT");
```

```
digitalWrite(GREEN_LED, HIGH);  
digitalWrite(YELLOW_LED, LOW);  
digitalWrite(RED_LED, LOW);  
delay(5000);
```

```
if (digitalRead(BUTTON_PIN) == LOW) {  
    pedestrianSequence();  
    return;  
}
```

```
Serial.println("YELLOW LIGHT");  
digitalWrite(GREEN_LED, LOW);  
digitalWrite(YELLOW_LED, HIGH);  
digitalWrite(RED_LED, LOW);  
delay(2000);
```

```
Serial.println("RED LIGHT");  
digitalWrite(GREEN_LED, LOW);  
digitalWrite(YELLOW_LED, LOW);  
digitalWrite(RED_LED, HIGH);  
delay(5000);  
}
```

```
void pedestrianSequence() {  
    Serial.println("PEDESTRIAN REQUEST DETECTED");
```

```
    digitalWrite(GREEN_LED, LOW);  
    digitalWrite(YELLOW_LED, HIGH);  
    delay(2000);
```

```
    digitalWrite(YELLOW_LED, LOW);  
    digitalWrite(RED_LED, HIGH);
```

```
    Serial.println("PEDESTRIAN MAY CROSS");  
    delay(5000);
```

```
    digitalWrite(RED_LED, LOW);
```

```
    Serial.println("RETURNING TO NORMAL TRAFFIC SEQUENCE");
```

```
    delay(500);  
}
```

Diagram Code :-

```
{
  "version": 1,
  "author": "Piyush Bhamare",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-arduino-uno",
      "id": "uno",
      "top": 20,
      "left": 20,
      "attrs": {}
    },
    {
      "type": "wokwi-led",
      "id": "red",
      "top": 20,
      "left": 360,
      "attrs": { "color": "red" }
    },
    {
      "type": "wokwi-led",
      "id": "yellow",
      "top": 100,
      "left": 360,
      "attrs": { "color": "yellow" }
    },
    {
      "type": "wokwi-led",
      "id": "green",
      "top": 180,
      "left": 360,
      "attrs": { "color": "green" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": 40,
      "left": 460,
      "attrs": { "value": "220" }
    },
    {
```

```

    "type": "wokwi-resistor",
    "id": "r2",
    "top": 120,
    "left": 460,
    "attrs": { "value": "220" }
  },
  {
    "type": "wokwi-resistor",
    "id": "r3",
    "top": 200,
    "left": 460,
    "attrs": { "value": "220" }
  },
  {
    "type": "wokwi-pushbutton",
    "id": "button",
    "top": 300,
    "left": 360,
    "attrs": {}
  }
],
"connections": [
  [ "uno:10", "red:A", "green", [] ],
  [ "red:C", "r1:1", "black", [] ],
  [ "r1:2", "uno:GND.1", "black", [] ],

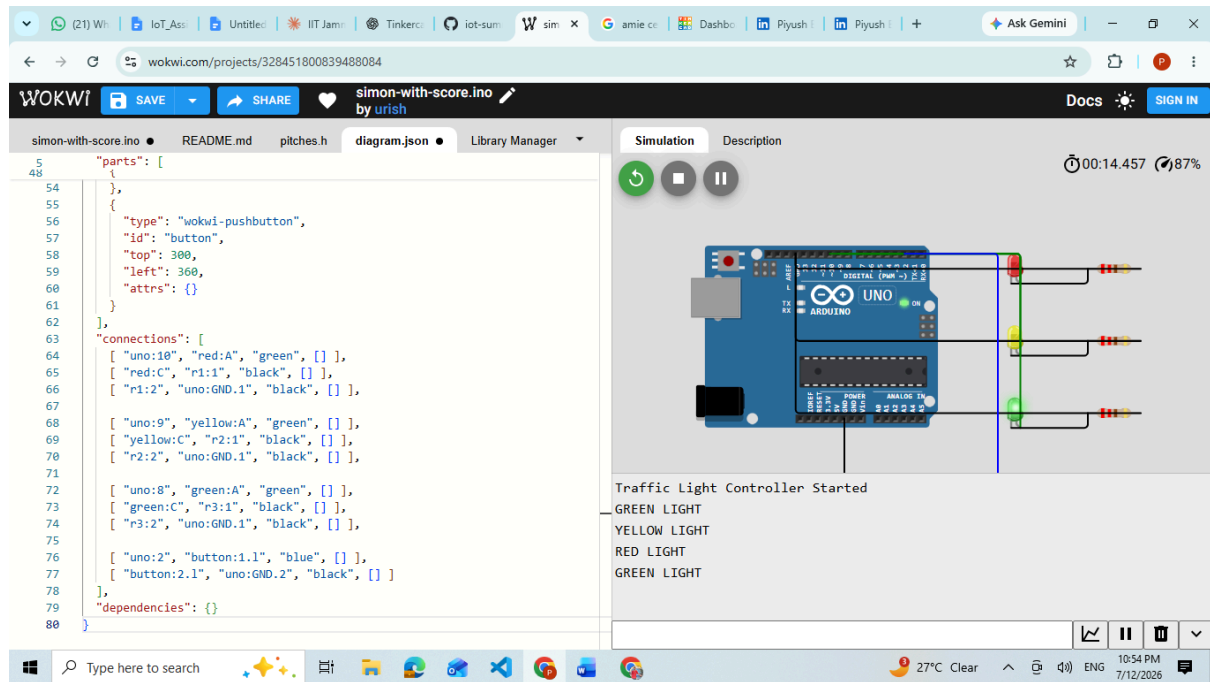
  [ "uno:9", "yellow:A", "green", [] ],
  [ "yellow:C", "r2:1", "black", [] ],
  [ "r2:2", "uno:GND.1", "black", [] ],

  [ "uno:8", "green:A", "green", [] ],
  [ "green:C", "r3:1", "black", [] ],
  [ "r3:2", "uno:GND.1", "black", [] ],

  [ "uno:2", "button:1.1", "blue", [] ],
  [ "button:2.1", "uno:GND.2", "black", [] ]
],
"dependencies": {}
}

```

Circuit Diagram :-



Q15. Digital Piano using Buzzer

Main Code :-

```

const int BUTTON_C = 2;
const int BUTTON_D = 3;
const int BUTTON_E = 4;
const int BUTTON_F = 5;

```

```

const int BUZZER = 8;

```

```

void setup() {
  pinMode(BUTTON_C, INPUT_PULLUP);
  pinMode(BUTTON_D, INPUT_PULLUP);
  pinMode(BUTTON_E, INPUT_PULLUP);
  pinMode(BUTTON_F, INPUT_PULLUP);

```

```

  pinMode(BUZZER, OUTPUT);

```

```

  Serial.begin(9600);
  Serial.println("Digital Piano Started");
}

```

```

void loop() {
  if (digitalRead(BUTTON_C) == LOW) {
    tone(BUZZER, 262);
    Serial.println("Note C");

```

```

}
else if (digitalRead(BUTTON_D) == LOW) {
  tone(BUZZER, 294);
  Serial.println("Note D");
}
else if (digitalRead(BUTTON_E) == LOW) {
  tone(BUZZER, 330);
  Serial.println("Note E");
}
else if (digitalRead(BUTTON_F) == LOW) {
  tone(BUZZER, 349);
  Serial.println("Note F");
}
else {
  noTone(BUZZER);
}

delay(50);
}

```

Diagram Code:-

```

{
  "version": 1,
  "author": "Piyush Bhamare",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 20, "left": 20, "attrs": {} },

    { "type": "wokwi-pushbutton", "id": "b1", "top": 20, "left": 350, "attrs": {} },
    { "type": "wokwi-pushbutton", "id": "b2", "top": 100, "left": 350, "attrs": {} },
    { "type": "wokwi-pushbutton", "id": "b3", "top": 180, "left": 350, "attrs": {} },
    { "type": "wokwi-pushbutton", "id": "b4", "top": 260, "left": 350, "attrs": {} },

    { "type": "wokwi-buzzer", "id": "buzzer", "top": 350, "left": 350, "attrs": {} }
  ],
  "connections": [
    [ "uno:2", "b1:1.l", "green", [] ],
    [ "b1:2.l", "uno:GND.1", "black", [] ],

    [ "uno:3", "b2:1.l", "green", [] ],
    [ "b2:2.l", "uno:GND.1", "black", [] ],

    [ "uno:4", "b3:1.l", "green", [] ],
    [ "b3:2.l", "uno:GND.1", "black", [] ],

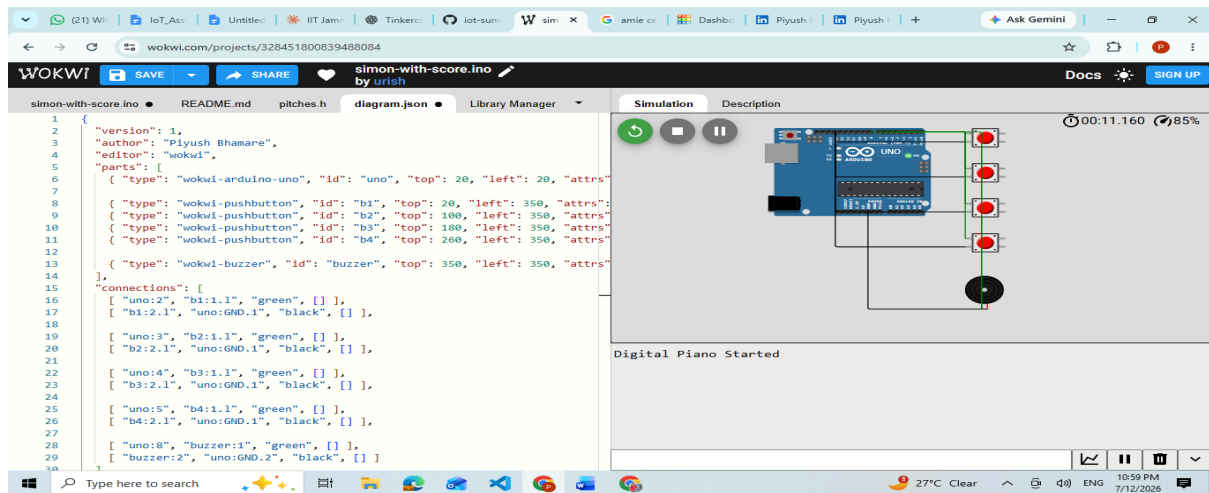
    [ "uno:5", "b4:1.l", "green", [] ],

```

```
[ "b4:2.1", "uno:GND.1", "black", [] ],

[ "uno:8", "buzzer:1", "green", [] ],
[ "buzzer:2", "uno:GND.2", "black", [] ]
],
"dependencies": {}
}
```

Circuit Diagram :-



Q16. Serial Command Interface

Main Code :-

```
const int LED_PIN = 13;
```

```
String command = "";
```

```
void setup() {
  pinMode(LED_PIN, OUTPUT);
```

```
  digitalWrite(LED_PIN, LOW);
```

```
  Serial.begin(9600);
```

```
  Serial.println("Serial LED Controller Started");
```

```
  Serial.println("Commands: ON, OFF, STATUS");
```

```
}
```

```
void loop() {
  if (Serial.available() > 0) {
    command = Serial.readStringUntil('\n');
```

```
    command.trim();
```

```
    command.toUpperCase();
```

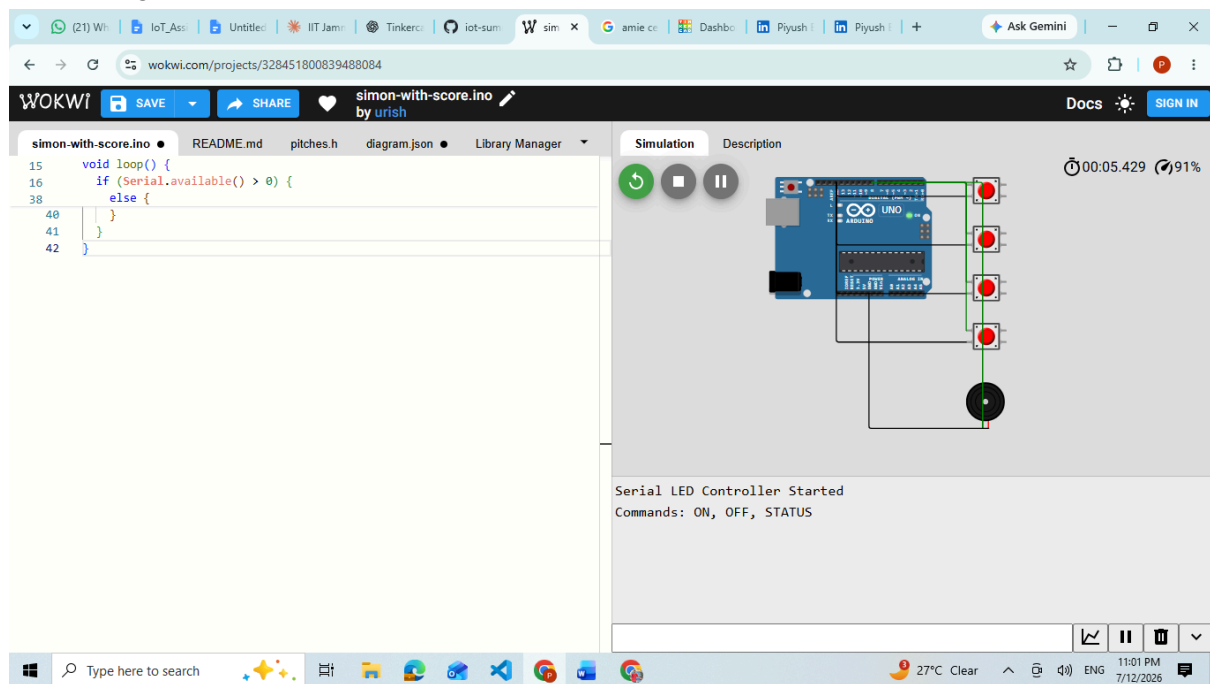
```
    if (command == "ON") {
```

```

    digitalWrite(LED_PIN, HIGH);
    Serial.println("LED TURNED ON");
}
else if (command == "OFF") {
    digitalWrite(LED_PIN, LOW);
    Serial.println("LED TURNED OFF");
}
else if (command == "STATUS") {
    if (digitalRead(LED_PIN) == HIGH) {
        Serial.println("LED STATUS: ON");
    }
    else {
        Serial.println("LED STATUS: OFF");
    }
}
else {
    Serial.println("INVALID COMMAND");
}
}
}
}

```

Circuit Diagram :-



Q17. PWM Fading Night Light

Main Code :-

```

const int LDR_PIN = A0;
const int LED_PIN = 9;

```



```

const int THRESHOLD = 500;

void setup() {
  pinMode(LED_PIN, OUTPUT);

  Serial.begin(9600);

  Serial.println("Automatic Night Light Started");
}

void loop() {
  int lightValue = analogRead(LDR_PIN);

  Serial.print("Light Value: ");
  Serial.print(lightValue);

  if (lightValue < THRESHOLD) {
    digitalWrite(LED_PIN, HIGH);

    Serial.println(" | DARK | LED ON");
  }
  else {
    digitalWrite(LED_PIN, LOW);

    Serial.println(" | BRIGHT | LED OFF");
  }

  delay(500);
}

```

Diagram Code :-

```

{
  "version": 1,
  "author": "Piyush Bhamare",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-arduino-uno",
      "id": "uno",
      "top": 20,
      "left": 20,
      "attrs": {}
    },
    {
      "type": "wokwi-photoresistor-sensor",
      "id": "ldr",
      "top": 30,

```

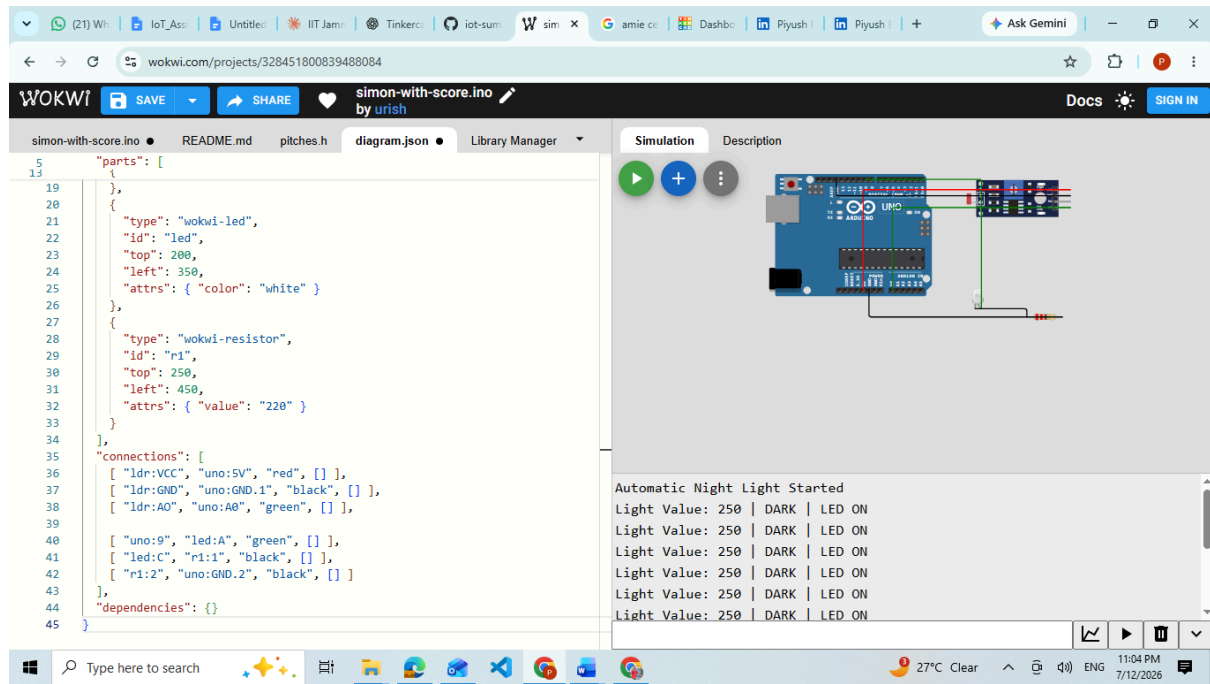
```

    "left": 350,
    "attrs": {}
  },
  {
    "type": "wokwi-led",
    "id": "led",
    "top": 200,
    "left": 350,
    "attrs": { "color": "white" }
  },
  {
    "type": "wokwi-resistor",
    "id": "r1",
    "top": 250,
    "left": 450,
    "attrs": { "value": "220" }
  }
],
"connections": [
  [ "ldr:VCC", "uno:5V", "red", [] ],
  [ "ldr:GND", "uno:GND.1", "black", [] ],
  [ "ldr:AO", "uno:A0", "green", [] ],

  [ "uno:9", "led:A", "green", [] ],
  [ "led:C", "r1:1", "black", [] ],
  [ "r1:2", "uno:GND.2", "black", [] ]
],
"dependencies": {}
}

```

Circuit Diagram :-



Q18. State Machine Implementation

Main Code :-

```

enum VendingState {
    IDLE,
    MONEY_INSERTED,
    PRODUCT_SELECTED,
    DISPENSING
};

VendingState currentState = IDLE;

String command = "";

void setup() {
    Serial.begin(9600);

    Serial.println("Vending Machine Started");
    Serial.println("Commands: MONEY, SELECT, CANCEL");

    displayState();
}

void loop() {
    if (Serial.available() > 0) {
        command = Serial.readStringUntil('\n');

        command.trim();
    }
}

```

```

    command.toUpperCase();

    processCommand(command);
}
}

void processCommand(String cmd) {
    switch (currentState) {

        case IDLE:

            if (cmd == "MONEY") {
                currentState = MONEY_INSERTED;

                Serial.println("Money Accepted");
            }
            else {
                Serial.println("Insert Money First");
            }

            break;

        case MONEY_INSERTED:

            if (cmd == "SELECT") {
                currentState = PRODUCT_SELECTED;

                Serial.println("Product Selected");
            }
            else if (cmd == "CANCEL") {
                currentState = IDLE;

                Serial.println("Transaction Cancelled");
                Serial.println("Money Returned");
            }
            else {
                Serial.println("Select Product or Cancel");
            }

            break;

        case PRODUCT_SELECTED:

            Serial.println("Preparing Product");

            currentState = DISPENSING;

            dispenseProduct();

```

```

        break;

    case DISPENSING:

        dispenseProduct();

        break;
    }

    displayState();
}

void dispenseProduct() {
    Serial.println("Dispensing Product...");

    delay(2000);

    Serial.println("Product Dispensed");

    currentState = IDLE;
}

void displayState() {
    Serial.print("Current State: ");

    switch (currentState) {
        case IDLE:
            Serial.println("IDLE");
            break;

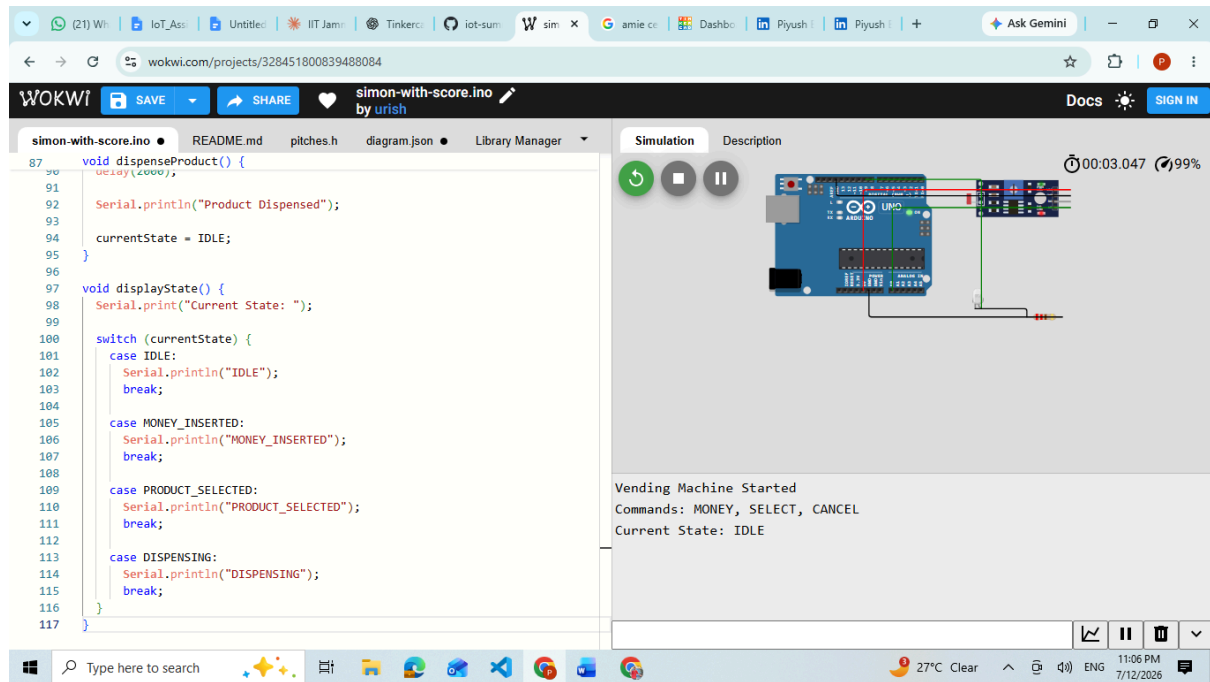
        case MONEY_INSERTED:
            Serial.println("MONEY_INSERTED");
            break;

        case PRODUCT_SELECTED:
            Serial.println("PRODUCT_SELECTED");
            break;

        case DISPENSING:
            Serial.println("DISPENSING");
            break;
    }
}

```

Circuit Diagram :-



Q19. What is the difference between analogWrite() and analogRead() in Arduino? What is PWM and why is it used? Give a practical IoT example for each. (2 marks)

Pulse Width Modulation, commonly called PWM, is a technique used to simulate different analog output levels using a digital signal.

A digital output can normally have only two states: HIGH and LOW.

In Arduino UNO, HIGH represents approximately 5 volts, while LOW represents approximately 0 volts.

PWM rapidly switches a digital output pin between HIGH and LOW.

By changing the percentage of time for which the signal remains HIGH during one complete cycle, the average output voltage and power supplied to the connected device can be controlled.

This percentage is called the duty cycle.

A 0% duty cycle means the signal is always LOW.

A 25% duty cycle means the signal is HIGH for 25% of each cycle.

A 50% duty cycle means the signal is HIGH for half of each cycle.

A 75% duty cycle means the signal is HIGH for 75% of each cycle.

A 100% duty cycle means the signal is always HIGH.

Arduino UNO contains six PWM-capable pins:

D3, D5, D6, D9, D10, and D11.

These pins are identified by the tilde (~) symbol on the Arduino UNO board.

The `analogWrite()` function is used to generate PWM output.

Syntax:

```
analogWrite(pin, value);
```

The value can range from 0 to 255.

`analogWrite(pin, 0)` represents approximately 0% duty cycle.

`analogWrite(pin, 127)` represents approximately 50% duty cycle.

`analogWrite(pin, 255)` represents approximately 100% duty cycle.

PWM can be used for LED brightness control, DC motor speed control, heater power control, and other applications requiring variable average power.

It is important to understand that `analogWrite()` on the Arduino UNO does not generate a true analog voltage. It generates a digital PWM waveform whose average effect can be controlled by changing the duty cycle.

CONCLUSION

PWM is an important technique for controlling the average power supplied to electronic devices using digital output pins.

The Arduino `analogWrite()` function generates PWM signals with values ranging from 0 to 255.

PWM is widely used for LED brightness control, motor speed control, and other embedded system applications.

Q20. Explain the `setup()` and `loop()` functions in Arduino. What happens if you put a long `delay()` inside `loop()`? How does this affect sensor reading responsiveness? What is the non-blocking alternative? (2 marks)

Every standard Arduino program contains two important functions:

```
setup()
```

```
loop()
```

These functions control the execution structure of an Arduino program.

setup() FUNCTION

The setup() function executes only once when the Arduino board is powered ON, reset, or a new program is uploaded.

It is mainly used for initialization.

Common operations performed inside setup() include configuring pin modes, starting Serial communication, initializing sensors, starting communication protocols, and setting initial output states.

Example:

```
void setup()
{
  pinMode(13, OUTPUT);

  Serial.begin(9600);
}
```

In this example, pin 13 is configured as an output and Serial communication is initialized at 9600 bits per second.

The setup() function executes only once.

loop() FUNCTION

The loop() function executes continuously after the setup() function has completed.

When Arduino reaches the end of loop(), it automatically starts executing loop() again.

Therefore, the loop() function is used for repeated operations such as reading sensors, checking buttons, controlling actuators, processing commands, transmitting data, and updating displays.

Example:

```
void loop()
{
  digitalWrite(13, HIGH);

  delay(1000);

  digitalWrite(13, LOW);

  delay(1000);
}
```


The LED continuously turns ON for one second and OFF for one second because the loop() function repeats indefinitely.

EXECUTION FLOW

Power ON or Reset



Microcontroller Initialization



setup() executes once



loop() starts



Statements inside loop() execute



End of loop() reached



loop() starts again



Process repeats continuously

DIFFERENCE BETWEEN setup() AND loop()

setup() executes only once, while loop() executes continuously.

setup() is used for initialization, while loop() is used for repeated program operations.

setup() commonly contains pinMode(), Serial.begin(), and sensor initialization instructions.

loop() commonly contains sensor readings, button checks, actuator control, communication operations, and program logic.

CONCLUSION

The setup() and loop() functions form the basic structure of an Arduino program.

The setup() function initializes the system and executes once.

The loop() function executes continuously and performs the repeated operations required by the embedded system.

Q21. Environmental Monitoring Station (DHT11)

Main Code :-

```
#include <DHT.h>
```

```
#define DHTPIN 2
```

```
#define DHTTYPE DHT11
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    dht.begin();
```

```
    Serial.println("DHT11 Temperature and Humidity Monitoring System");
```

```
    Serial.println("-----");
```

```
}
```

```
void loop() {
```

```
    delay(2000);
```

```
    float humidity = dht.readHumidity();
```

```
    float temperature = dht.readTemperature();
```

```
    if (isnan(humidity) || isnan(temperature)) {
```

```
        Serial.println("ERROR: Failed to read data from DHT11 sensor.");
```

```
        return;
```

```
    }
```

```
    Serial.print("Temperature: ");
```

```
    Serial.print(temperature);
```

```
    Serial.print(" °C");
```

```
    Serial.print(" | Humidity: ");
```

```
    Serial.print(humidity);
```

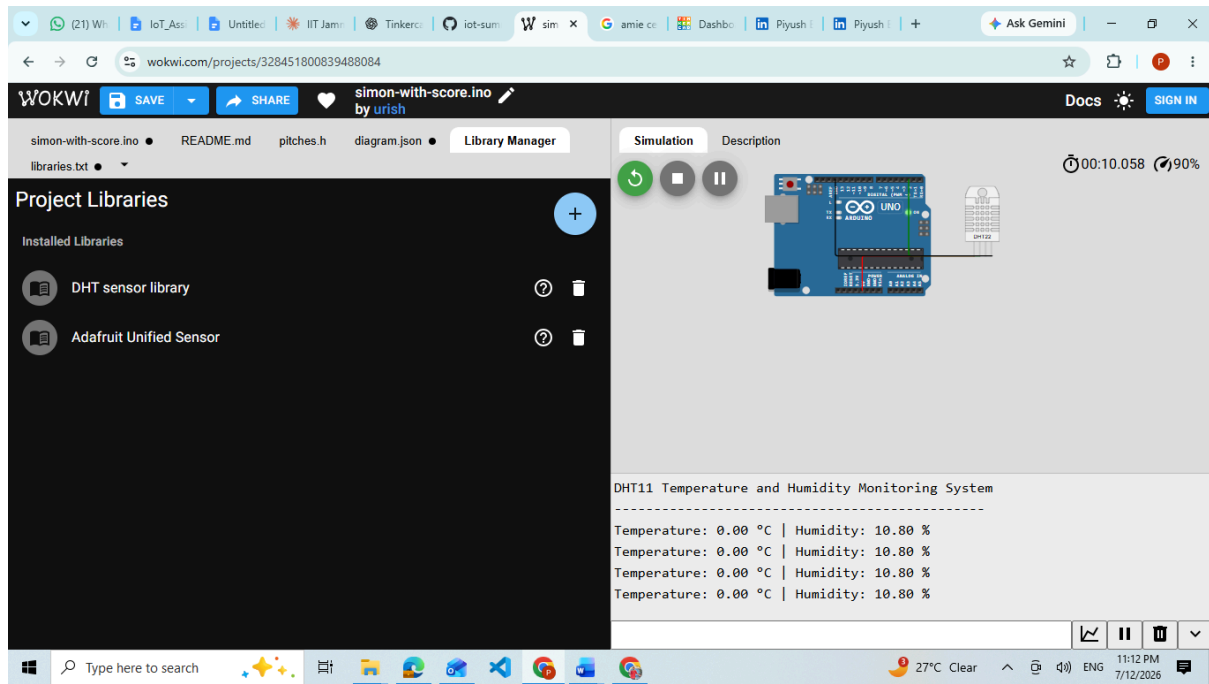
```
    Serial.println(" %");
```

```
}
```

Diagram Code :-

```
{
  "version": 1,
  "author": "Piyush Bhamare",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-arduino-uno",
      "id": "uno",
      "top": 20,
      "left": 20,
      "attrs": {}
    },
    {
      "type": "wokwi-dht22",
      "id": "dht",
      "top": 40,
      "left": 350,
      "attrs": {
        "temperature": "25",
        "humidity": "60"
      }
    }
  ],
  "connections": [
    ["dht:VCC", "uno:5V", "red", []],
    ["dht:SDA", "uno:2", "green", []],
    ["dht:GND", "uno:GND.1", "black", []]
  ],
  "dependencies": {}
}
```

Circuit Diagram :-



Q22. Ultrasonic Parking Sensor

Main Code :-

```
const int TRIG_PIN = 9;
const int ECHO_PIN = 10;
```

```
const int YELLOW_LED = 6;
const int RED_LED = 7;
```

```
const int BUZZER_PIN = 5;
```

```
const float EMERGENCY_DISTANCE = 20.0;
```

```
unsigned long previousBeepTime = 0;
unsigned long previousFlashTime = 0;
```

```
bool beepState = false;
bool flashState = false;
```

```
float readDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
```

```
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
```

```
  digitalWrite(TRIG_PIN, LOW);
```

```
  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
```

```

    if (duration == 0) {
        return -1;
    }

    return duration * 0.0343 / 2.0;
}

void resetOutputs() {
    digitalWrite(YELLOW_LED, LOW);
    digitalWrite(RED_LED, LOW);

    noTone(BUZZER_PIN);

    beepState = false;
}

void intermittentAlarm(unsigned long interval) {
    unsigned long currentTime = millis();

    if (currentTime - previousBeepTime >= interval) {
        previousBeepTime = currentTime;

        beepState = !beepState;

        if (beepState) {
            tone(BUZZER_PIN, 1000);
        }
        else {
            noTone(BUZZER_PIN);
        }
    }
}

void emergencyAlarm() {
    unsigned long currentTime = millis();

    tone(BUZZER_PIN, 1500);

    if (currentTime - previousFlashTime >= 200) {
        previousFlashTime = currentTime;

        flashState = !flashState;

        digitalWrite(RED_LED, flashState);
    }
}

```

```

void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  pinMode(YELLOW_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);

  pinMode(BUZZER_PIN, OUTPUT);

  Serial.begin(9600);

  Serial.println("ULTRASONIC PARKING SENSOR STARTED");
}

void loop() {
  float distance = readDistance();

  if (distance < 0) {
    Serial.println("ERROR: NO ECHO RECEIVED");
    resetOutputs();
    return;
  }

  Serial.print("[");
  Serial.print(millis());
  Serial.print(" ms] Distance: ");
  Serial.print(distance);
  Serial.print(" cm | ");

  if (distance > 100) {
    resetOutputs();

    Serial.println("ZONE: SAFE");
  }

  else if (distance >= 50) {
    digitalWrite(YELLOW_LED, HIGH);
    digitalWrite(RED_LED, LOW);

    intermittentAlarm(500);

    Serial.println("ZONE: WARNING");
  }

  else if (distance >= EMERGENCY_DISTANCE) {
    digitalWrite(YELLOW_LED, LOW);
    digitalWrite(RED_LED, HIGH);
  }
}

```

```

intermittentAlarm(200);

Serial.println("ZONE: DANGER");
}

else {
    digitalWrite(YELLOW_LED, LOW);

    emergencyAlarm();

    Serial.println("ZONE: EMERGENCY");
}

delay(50);
}

```

Circuit Diagram :-

The screenshot displays the Wokwi online IDE interface. On the left, the code editor shows the following JSON configuration for an ultrasonic sensor:

```

5
12  },
13
14  {
15    "type": "woki-hc-sr04",
16    "id": "ultrasonic",
17    "top": 30,
18    "left": 350,
19    "attrs": {
20      "distance": "100"
21    }
22  },
23  "connections": [
24    ["ultrasonic:VCC", "uno:5V", "red", []],
25    ["ultrasonic:TRIG", "uno:9", "green", []],
26    ["ultrasonic:ECHO", "uno:10", "blue", []],
27    ["ultrasonic:GND", "uno:GND.1", "black", []]
28  ],
29  "dependencies": {}
30 }

```

On the right, the simulation window shows a visual representation of the Arduino Uno and the ultrasonic sensor. Below the visual, the output log displays the following distance readings:

```

Distance: 100.36 cm
Distance: 100.28 cm
Distance: 100.26 cm
Distance: 100.28 cm
Distance: 100.28 cm
Distance: 100.36 cm
Distance: 100.28 cm

```

Q23. Smart Street Light (LDR + PIR)

Main Code :-

```
const int LDR_PIN = A0;
const int PIR_PIN = 2;
const int LED_PIN = 9;

const int NIGHT_THRESHOLD = 500;

const int DIM_BRIGHTNESS = 51; // 20% of 255
const int FULL_BRIGHTNESS = 255;

const unsigned long FULL_LIGHT_DURATION = 30000;

unsigned long lastMotionTime = 0;

bool motionTimerActive = false;

String previousState = "";

void printEvent(String eventMessage) {
  Serial.print("[");
  Serial.print(millis());
  Serial.print(" ms] ");
  Serial.println(eventMessage);
}

void updateState(String newState) {
  if (newState != previousState) {
    printEvent(newState);

    previousState = newState;
  }
}

void setup() {
  pinMode(PIR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);

  Serial.begin(9600);

  Serial.println("SMART STREET LIGHT STARTED");
}

void loop() {
  int lightValue = analogRead(LDR_PIN);

  bool motionDetected = digitalRead(PIR_PIN);

  bool nightTime = lightValue < NIGHT_THRESHOLD;
```



```

if (!nightTime) {
    analogWrite(LED_PIN, 0);

    motionTimerActive = false;

    updateState("DAYLIGHT | LED OFF");
}

else {
    if (motionDetected) {
        lastMotionTime = millis();

        motionTimerActive = true;

        printEvent("MOTION DETECTED");
    }

    if (motionTimerActive &&
        millis() - lastMotionTime < FULL_LIGHT_DURATION) {

        analogWrite(LED_PIN, FULL_BRIGHTNESS);

        updateState("NIGHT + MOTION | LED 100%");
    }

    else {
        motionTimerActive = false;

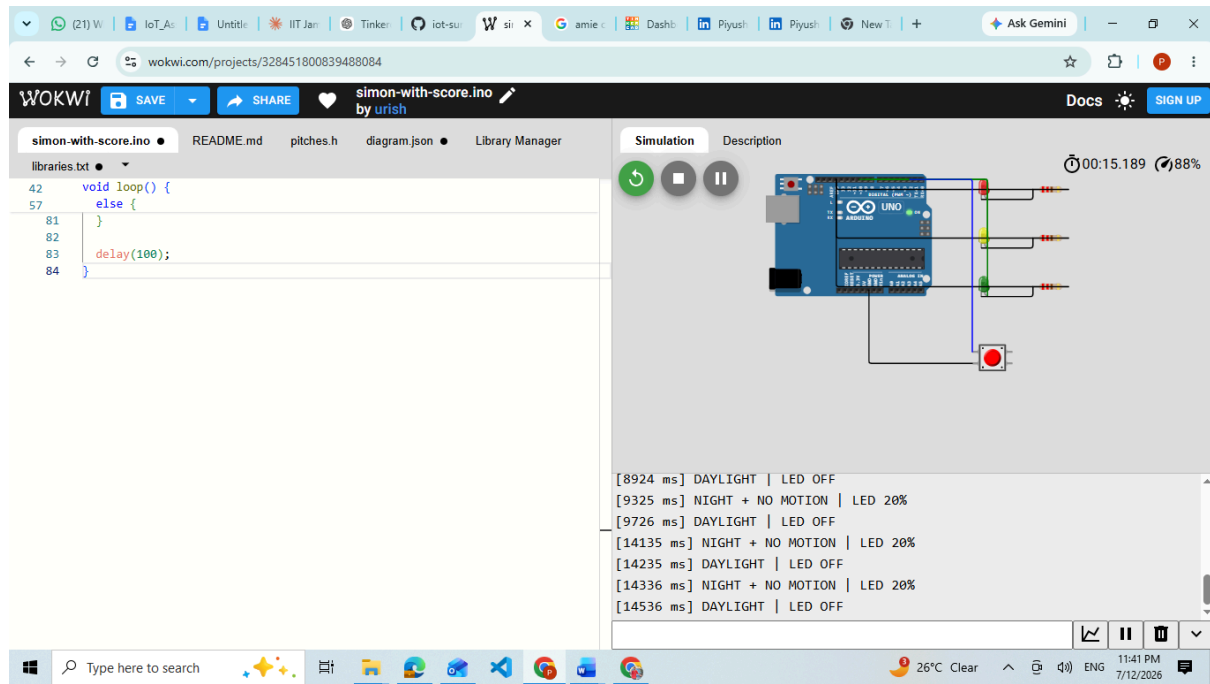
        analogWrite(LED_PIN, DIM_BRIGHTNESS);

        updateState("NIGHT + NO MOTION | LED 20%");
    }
}

delay(100);
}

```

Circuit Diagram :-



Q24. Multi-Sensor Data Logger

Main code :-

```
#include <DHT.h>
```

```
#define DHT_PIN 2
```

```
#define DHT_TYPE DHT11
```

```
const int LDR_PIN = A0;
```

```
const int TRIG_PIN = 9;
```

```
const int ECHO_PIN = 10;
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
unsigned long sampleNumber = 0;
```

```
float readDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
```

```
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
```

```
  digitalWrite(TRIG_PIN, LOW);
```

```
  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
```

```

    if (duration == 0) {
        return -1;
    }

    return duration * 0.0343 / 2.0;
}

void setup() {
    Serial.begin(9600);

    dht.begin();

    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);

    Serial.println("sample,timestamp_ms,temperature_c,humidity_percent,ldr_value,distance_cm");
}

void loop() {
    sampleNumber++;

    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    int ldrValue = analogRead(LDR_PIN);

    float distance = readDistance();

    Serial.print(sampleNumber);
    Serial.print(",");

    Serial.print(millis());
    Serial.print(",");

    if (isnan(temperature)) {
        Serial.print("ERROR");
    }
    else {
        Serial.print(temperature);
    }

    Serial.print(",");

    if (isnan(humidity)) {
        Serial.print("ERROR");
    }
    else {
        Serial.print(humidity);
    }
}

```

```

}

Serial.print(",");

Serial.print(ldrValue);
Serial.print(",");

if (distance < 0) {
    Serial.println("ERROR");
}
else {
    Serial.println(distance);
}

delay(5000);
}

```

Circuit Diagram :-

The screenshot displays the Wokwi online IDE interface. On the left, the code editor shows a C++ program snippet for a sensor-based system. The code includes logic for handling humidity and distance readings, printing them to the serial monitor, and handling errors. On the right, the simulation window shows a circuit diagram of an Arduino Uno connected to a sensor module. The sensor module has several pins connected to the Arduino's digital pins. Below the circuit diagram, the serial monitor displays the output of the code, showing a series of data points including sample numbers, timestamps, temperature, humidity, ldr values, and distance measurements. The simulation is running, as indicated by the play button and the elapsed time of 00:08.095.

Q25. Servo Motor Sweep

Main Code :-

```
#include <Servo.h>
```

```

const int POT_PIN = A0;
const int SERVO_PIN = 9;
const int SWEEP_BUTTON = 2;

```

```
Servo servoMotor;
```

```

bool previousButtonState = HIGH;

void performFullSweep() {
  Serial.println("FULL SWEEP STARTED");

  for (int angle = 0; angle <= 180; angle++) {
    servoMotor.write(angle);

    delay(15);
  }

  for (int angle = 180; angle >= 0; angle--) {
    servoMotor.write(angle);

    delay(15);
  }

  Serial.println("FULL SWEEP COMPLETED");
}

void setup() {
  servoMotor.attach(SERVO_PIN);

  pinMode(SWEEP_BUTTON, INPUT_PULLUP);

  Serial.begin(9600);

  Serial.println("SERVO CONTROL SYSTEM STARTED");
}

void loop() {
  bool currentButtonState = digitalRead(SWEEP_BUTTON);

  if (previousButtonState == HIGH &&
      currentButtonState == LOW) {

    performFullSweep();

    delay(200);
  }

  previousButtonState = currentButtonState;

  int potentiometerValue = analogRead(POT_PIN);

  int servoAngle = map(
    potentiometerValue,

```

```

0,
1023,
0,
180
);

```

```

servoMotor.write(servoAngle);

```

```

Serial.print("POTENTIOMETER: ");
Serial.print(potentiometerValue);

```

```

Serial.print(" | SERVO ANGLE: ");
Serial.println(servoAngle);

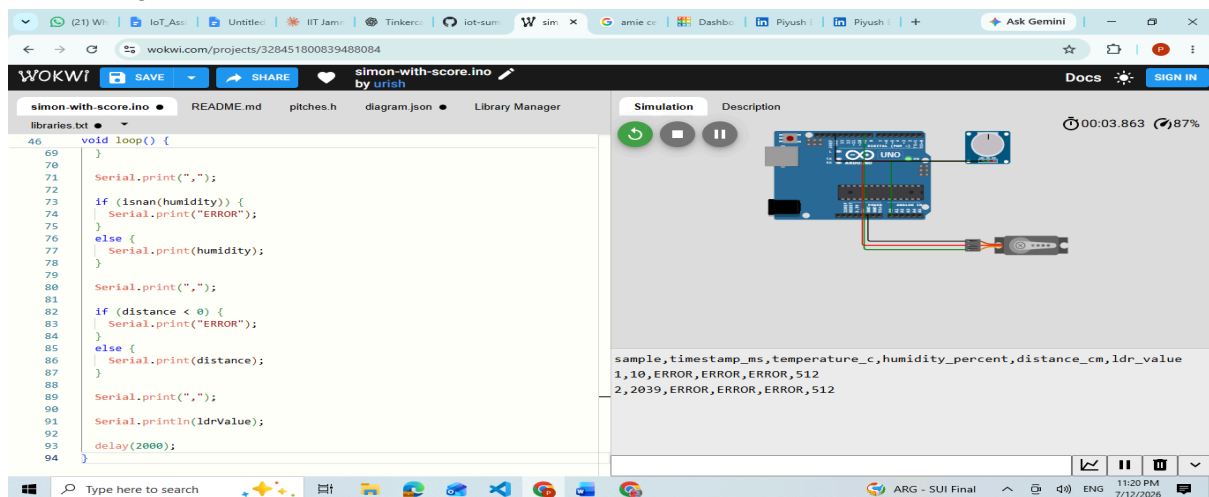
```

```

delay(100);
}

```

Circuit Diagram :-



Q26. DC Motor Speed Control with L298N

Main Code :-

```
const int ENA_PIN = 9;
const int IN1_PIN = 7;
const int IN2_PIN = 8;

const int POT_PIN = A0;

const int DIRECTION_BUTTON = 2;
const int START_STOP_BUTTON = 3;

bool motorRunning = false;
bool clockwiseDirection = true;

bool previousDirectionButton = HIGH;
bool previousStartStopButton = HIGH;

unsigned long lastDirectionPress = 0;
unsigned long lastStartStopPress = 0;

const unsigned long DEBOUNCE_DELAY = 200;

void updateDirection() {
  if (clockwiseDirection) {
    digitalWrite(IN1_PIN, HIGH);
    digitalWrite(IN2_PIN, LOW);
  }
  else {
    digitalWrite(IN1_PIN, LOW);
    digitalWrite(IN2_PIN, HIGH);
  }
}

void setup() {
  pinMode(ENA_PIN, OUTPUT);
  pinMode(IN1_PIN, OUTPUT);
  pinMode(IN2_PIN, OUTPUT);

  pinMode(DIRECTION_BUTTON, INPUT_PULLUP);
  pinMode(START_STOP_BUTTON, INPUT_PULLUP);

  Serial.begin(9600);

  updateDirection();

  analogWrite(ENA_PIN, 0);

  Serial.println("DC MOTOR CONTROL SYSTEM STARTED");
```

```

Serial.println("-----");
}

void loop() {
  bool currentDirectionButton =
    digitalRead(DIRECTION_BUTTON);

  bool currentStartStopButton =
    digitalRead(START_STOP_BUTTON);

  if (previousDirectionButton == HIGH &&
      currentDirectionButton == LOW &&
      millis() - lastDirectionPress >= DEBOUNCE_DELAY) {

    lastDirectionPress = millis();

    clockwiseDirection = !clockwiseDirection;

    updateDirection();

    Serial.print("DIRECTION CHANGED: ");

    if (clockwiseDirection) {
      Serial.println("CLOCKWISE");
    }
    else {
      Serial.println("ANTICLOCKWISE");
    }
  }

  if (previousStartStopButton == HIGH &&
      currentStartStopButton == LOW &&
      millis() - lastStartStopPress >= DEBOUNCE_DELAY) {

    lastStartStopPress = millis();

    motorRunning = !motorRunning;

    if (motorRunning) {
      Serial.println("MOTOR STARTED");
    }
    else {
      Serial.println("MOTOR STOPPED");
    }
  }

  previousDirectionButton = currentDirectionButton;
  previousStartStopButton = currentStartStopButton;
}

```



```
int potentiometerValue = analogRead(POT_PIN);
```

```
int motorSpeed = map(  
  potentiometerValue,  
  0,  
  1023,  
  0,  
  255  
);
```

```
if (motorRunning) {  
  analogWrite(ENA_PIN, motorSpeed);  
}  
else {  
  analogWrite(ENA_PIN, 0);  
}
```

```
Serial.print("POT VALUE: ");  
Serial.print(potentiometerValue);
```

```
Serial.print(" | SPEED PWM: ");
```

```
if (motorRunning) {  
  Serial.print(motorSpeed);  
}  
else {  
  Serial.print(0);  
}
```

```
Serial.print(" | MOTOR: ");
```

```
if (motorRunning) {  
  Serial.print("RUNNING");  
}  
else {  
  Serial.print("STOPPED");  
}
```

```
Serial.print(" | DIRECTION: ");
```

```
if (clockwiseDirection) {  
  Serial.println("CLOCKWISE");  
}  
else {  
  Serial.println("ANTICLOCKWISE");  
}
```

```
delay(500);  
}
```

Diagram Code :-

```
{  
  "version": 1,  
  "author": "Piyush Bhamare",  
  "editor": "wokwi",  
  "parts": [  
    {  
      "type": "wokwi-arduino-uno",  
      "id": "uno",  
      "top": 10,  
      "left": 10,  
      "attrs": {}  
    },  
    {  
      "type": "wokwi-l293d",  
      "id": "driver",  
      "top": 20,  
      "left": 360,  
      "attrs": {}  
    },  
    {  
      "type": "wokwi-dc-motor",  
      "id": "motor",  
      "top": 20,  
      "left": 650,  
      "attrs": {}  
    },  
    {  
      "type": "wokwi-potentiometer",  
      "id": "pot",  
      "top": 250,  
      "left": 360,  
      "attrs": {  
        "value": "512"  
      }  
    },  
    {  
      "type": "wokwi-pushbutton",  
      "id": "directionButton",  
      "top": 350,  
      "left": 350,  
      "attrs": {  
        "color": "blue",  
        "label": "DIRECTION"  
      }  
    }  
  ]  
}
```

```

    }
  },
  {
    "type": "wokwi-pushbutton",
    "id": "startButton",
    "top": 450,
    "left": 350,
    "attrs": {
      "color": "green",
      "label": "START/STOP"
    }
  }
],
"connections": [
  ["uno:9", "driver:1,2EN", "green", []],
  ["uno:7", "driver:1A", "blue", []],
  ["uno:8", "driver:2A", "blue", []],

  ["driver:1Y", "motor:1", "green", []],
  ["driver:2Y", "motor:2", "green", []],

  ["driver:VCC1", "uno:5V", "red", []],
  ["driver:VCC2", "uno:5V", "red", []],

  ["driver:GND1", "uno:GND.1", "black", []],
  ["driver:GND2", "uno:GND.1", "black", []],
  ["driver:GND3", "uno:GND.2", "black", []],
  ["driver:GND4", "uno:GND.2", "black", []],

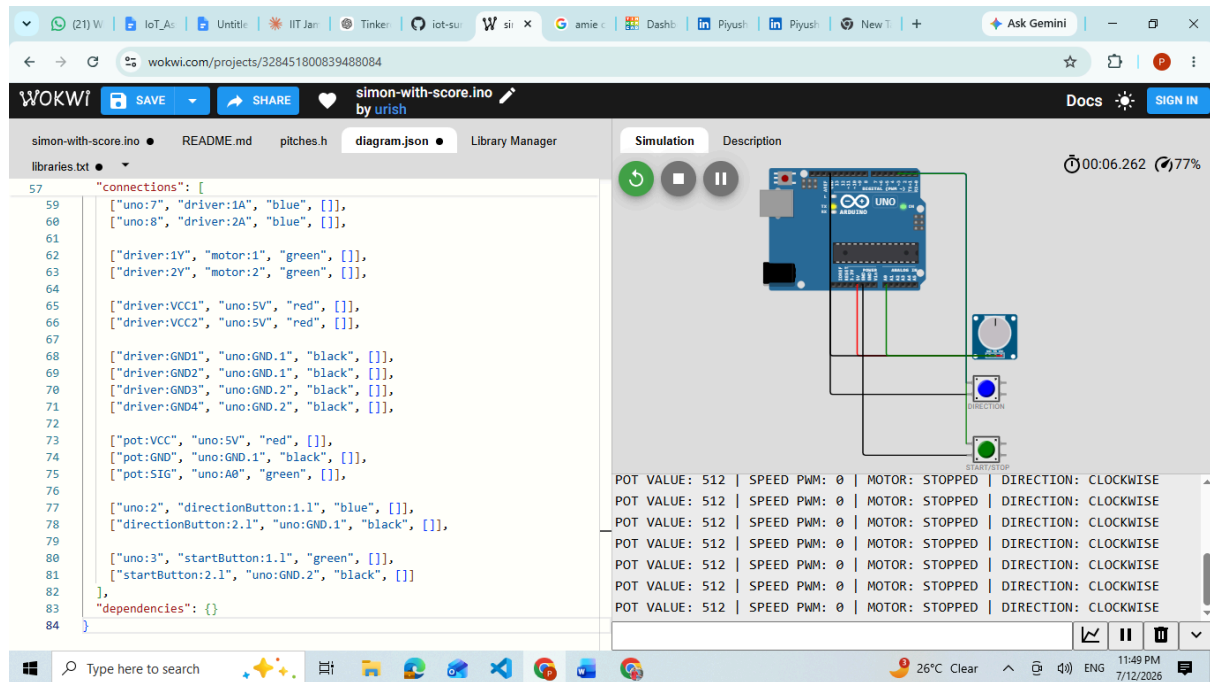
  ["pot:VCC", "uno:5V", "red", []],
  ["pot:GND", "uno:GND.1", "black", []],
  ["pot:SIG", "uno:A0", "green", []],

  ["uno:2", "directionButton:1.1", "blue", []],
  ["directionButton:2.1", "uno:GND.1", "black", []],

  ["uno:3", "startButton:1.1", "green", []],
  ["startButton:2.1", "uno:GND.2", "black", []]
],
"dependencies": {}
}

```

Circuit Diagram :-



Q27. Relay-Controlled AC Device Simulation

Main Code :-

```
#include <DHT.h>
```

```
#define DHT_PIN 2
```

```
#define DHT_TYPE DHT22
```

```
const int RELAY_PIN = 8;
```

```
const int OVERRIDE_BUTTON = 3;
```

```
const float TURN_ON_TEMPERATURE = 30.0;
```

```
const float TURN_OFF_TEMPERATURE = 27.0;
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
bool relayState = false;
```

```
bool manualMode = false;
```

```

bool previousButtonState = HIGH;

unsigned long lastButtonPress = 0;

const unsigned long DEBOUNCE_DELAY = 200;

void setRelay(bool newState) {
  if (newState != relayState) {
    relayState = newState;

    digitalWrite(RELAY_PIN, relayState);

    Serial.print("[");
    Serial.print(millis());
    Serial.print(" ms] RELAY STATE CHANGED: ");

    if (relayState) {
      Serial.println("ON");
    }
    else {
      Serial.println("OFF");
    }
  }
}

void setup() {
  pinMode(RELAY_PIN, OUTPUT);

  pinMode(OVERRIDE_BUTTON, INPUT_PULLUP);

  digitalWrite(RELAY_PIN, LOW);

  Serial.begin(9600);

  dht.begin();

  Serial.println("AUTOMATIC RELAY CONTROL SYSTEM STARTED");
  Serial.println("INITIAL MODE: AUTOMATIC");
  Serial.println("TURN ON TEMPERATURE: 30 C");
  Serial.println("TURN OFF TEMPERATURE: 27 C");
}

void loop() {
  bool currentButtonState =
    digitalRead(OVERRIDE_BUTTON);

  if (previousButtonState == HIGH &&

```

```

    currentButtonState == LOW &&
    millis() - lastButtonPress >= DEBOUNCE_DELAY) {

    lastButtonPress = millis();

    manualMode = !manualMode;

    Serial.print("[");
    Serial.print(millis());
    Serial.print(" ms] OPERATING MODE CHANGED: ");

    if (manualMode) {
        Serial.println("MANUAL OVERRIDE");
    }
    else {
        Serial.println("AUTOMATIC");
    }
}

previousButtonState = currentButtonState;

if (manualMode) {
    setRelay(true);
}
else {
    float temperature = dht.readTemperature();

    if (isnan(temperature)) {
        return;
    }

    if (!relayState &&
        temperature >= TURN_ON_TEMPERATURE) {

        setRelay(true);
    }

    else if (relayState &&
        temperature <= TURN_OFF_TEMPERATURE) {

        setRelay(false);
    }
}

delay(1000);
}

```

Diagram Code :-

```
{
  "version": 1,
  "author": "Piyush Bhamare",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-arduino-uno",
      "id": "uno",
      "top": 20,
      "left": 20,
      "attrs": {}
    },
    {
      "type": "wokwi-dht22",
      "id": "dht",
      "top": 20,
      "left": 380,
      "attrs": {
        "temperature": "25",
        "humidity": "60"
      }
    },
    {
      "type": "wokwi-relay-module",
      "id": "relay",
      "top": 200,
      "left": 380,
      "attrs": {}
    },
    {
      "type": "wokwi-led",
      "id": "acDevice",
      "top": 210,
      "left": 680,
      "attrs": {
        "color": "blue"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "deviceResistor",
      "top": 300,
      "left": 680,
      "attrs": {
        "value": "220"
      }
    }
  ],
}
```

```

{
  "type": "wokwi-pushbutton",
  "id": "overrideButton",
  "top": 400,
  "left": 380,
  "attrs": {
    "color": "red",
    "label": "MANUAL OVERRIDE"
  }
}
],
"connections": [
  ["dht:VCC", "uno:5V", "red", []],
  ["dht:SDA", "uno:2", "green", []],
  ["dht:GND", "uno:GND.1", "black", []],

  ["relay:VCC", "uno:5V", "red", []],
  ["relay:GND", "uno:GND.1", "black", []],
  ["relay:IN", "uno:8", "green", []],

  ["relay:COM", "uno:5V", "red", []],
  ["relay:NO", "acDevice:A", "green", []],

  ["acDevice:C", "deviceResistor:1", "black", []],
  ["deviceResistor:2", "uno:GND.2", "black", []],

  ["uno:3", "overrideButton:1.1", "blue", []],
  ["overrideButton:2.1", "uno:GND.2", "black", []]
],
"dependencies": {}
}

```

Circuit Diagram :-

WOKWI

SAVE

SHARE

simon-with-score.ino
by urish

Docs

SIGN IN

simon-with-score.ino

README.md

itches.h

diagram.json

Library Manager

libraries.txt

```

58   },
59   "connections": [
60     ["dht:VCC", "uno:5V", "red", []],
61     ["dht:SDA", "uno:2", "green", []],
62     ["dht:GND", "uno:GND.1", "black", []],
63
64     ["relay:VCC", "uno:5V", "red", []],
65     ["relay:GND", "uno:GND.1", "black", []],
66     ["relay:IN", "uno:8", "green", []],
67
68     ["relay:COM", "uno:5V", "red", []],
69     ["relay:NO", "acDevice:A", "green", []],
70
71     ["acDevice:C", "deviceResistor:1", "black", []],
72     ["deviceResistor:2", "uno:GND.2", "black", []],
73
74     ["uno:3", "overrideButton:1.1", "blue", []],
75     ["overrideButton:2.1", "uno:GND.2", "black", []]
76   ],
77   "dependencies": {}
78 }

```

Simulation

Description

00:03.031

86%

AUTOMATIC RELAY CONTROL SYSTEM STARTED

INITIAL MODE: AUTOMATIC

TURN ON TEMPERATURE: 30 C

TURN OFF TEMPERATURE: 27 C

Q28. Keypad + LCD Display

Main Code :-

```
#include <Keypad.h>
#include <LiquidCrystal.h>
const String CORRECT_PASSWORD = "1234";

const int MAX_ATTEMPTS = 3;

const unsigned long LOCKOUT_DURATION = 10000;

String enteredPassword = "";

int failedAttempts = 0;

bool systemLocked = false;

unsigned long lockoutStartTime = 0;

const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};

byte rowPins[ROWS] = {
  2, 3, 4, 5
};

byte colPins[COLS] = {
  6, 7, 8, 9
};

Keypad keypad = Keypad(
  makeKeymap(keys),
  rowPins,
  colPins,
  ROWS,
  COLS
);

LiquidCrystal lcd(
  A0,
```

```

    A1,
    A2,
    A3,
    A4,
    A5
);

void showReadyScreen() {
    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("Enter Password:");

    lcd.setCursor(0, 1);
}

void showMaskedPassword() {
    lcd.setCursor(0, 1);

    lcd.print("          ");

    lcd.setCursor(0, 1);

    for (unsigned int i = 0;
        i < enteredPassword.length();
        i++) {

        lcd.print("*");
    }
}

void setup() {
    lcd.begin(16, 2);

    Serial.begin(9600);

    showReadyScreen();

    Serial.println("PASSWORD ACCESS SYSTEM STARTED");
    Serial.println("PRESS * TO CLEAR PASSWORD");
    Serial.println("PRESS # TO SUBMIT PASSWORD");
}

void loop() {
    if (systemLocked) {
        unsigned long elapsedTime =
            millis() - lockoutStartTime;
    }
}

```

```

unsigned long remainingSeconds =
    (LOCKOUT_DURATION - elapsedTime + 999) / 1000;

lcd.setCursor(0, 1);
lcd.print("WAIT ");

if (remainingSeconds < 10) {
    lcd.print(" ");
}

lcd.print(remainingSeconds);
lcd.print(" SECONDS ");

if (elapsedTime >= LOCKOUT_DURATION) {
    systemLocked = false;

    failedAttempts = 0;

    enteredPassword = "";

    Serial.println("SYSTEM UNLOCKED");

    showReadyScreen();
}

return;
}

char key = keypad.getKey();

if (key) {
    if (key >= '0' && key <= '9') {
        if (enteredPassword.length() < 16) {
            enteredPassword += key;

            showMaskedPassword();
        }
    }

    else if (key == '*') {
        enteredPassword = "";

        showReadyScreen();

        Serial.println("PASSWORD ENTRY CLEARED");
    }

    else if (key == '#') {

```

```
if (enteredPassword == CORRECT_PASSWORD) {
    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("ACCESS GRANTED");

    Serial.println("ACCESS GRANTED");

    failedAttempts = 0;

    delay(2000);
}

else {
    failedAttempts++;

    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("ACCESS DENIED");

    lcd.setCursor(0, 1);
    lcd.print("ATTEMPT ");

    lcd.print(failedAttempts);

    lcd.print("/3");

    Serial.print("ACCESS DENIED | FAILED ATTEMPTS: ");
    Serial.println(failedAttempts);

    delay(2000);

    if (failedAttempts >= MAX_ATTEMPTS) {
        systemLocked = true;

        lockoutStartTime = millis();

        lcd.clear();

        lcd.setCursor(0, 0);
        lcd.print("SYSTEM LOCKED");

        lcd.setCursor(0, 1);
        lcd.print("WAIT 10 SECONDS");

        Serial.println("SYSTEM LOCKED FOR 10 SECONDS");
    }
}
```

```

    }

    enteredPassword = "";

    if (!systemLocked) {
        showReadyScreen();
    }
}
}
}
}

```

Diagram Code :-

```

{
  "version": 1,
  "author": "Piyush Bhamare",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-arduino-uno",
      "id": "uno",
      "top": 20,
      "left": 20,
      "attrs": {}
    },
    {
      "type": "wokwi-membrane-keypad",
      "id": "keypad",
      "top": 20,
      "left": 400,
      "attrs": {}
    },
    {
      "type": "wokwi-lcd1602",
      "id": "lcd",
      "top": 350,
      "left": 350,
      "attrs": {
        "pins": "full"
      }
    }
  ],
  "connections": [
    ["keypad:R1", "uno:2", "green", []],
    ["keypad:R2", "uno:3", "green", []],
    ["keypad:R3", "uno:4", "green", []],
    ["keypad:R4", "uno:5", "green", []],

```

```

["keypad:C1", "uno:6", "blue", []],
["keypad:C2", "uno:7", "blue", []],
["keypad:C3", "uno:8", "blue", []],
["keypad:C4", "uno:9", "blue", []],

["lcd:VSS", "uno:GND.1", "black", []],
["lcd:VDD", "uno:5V", "red", []],
["lcd:RW", "uno:GND.1", "black", []],

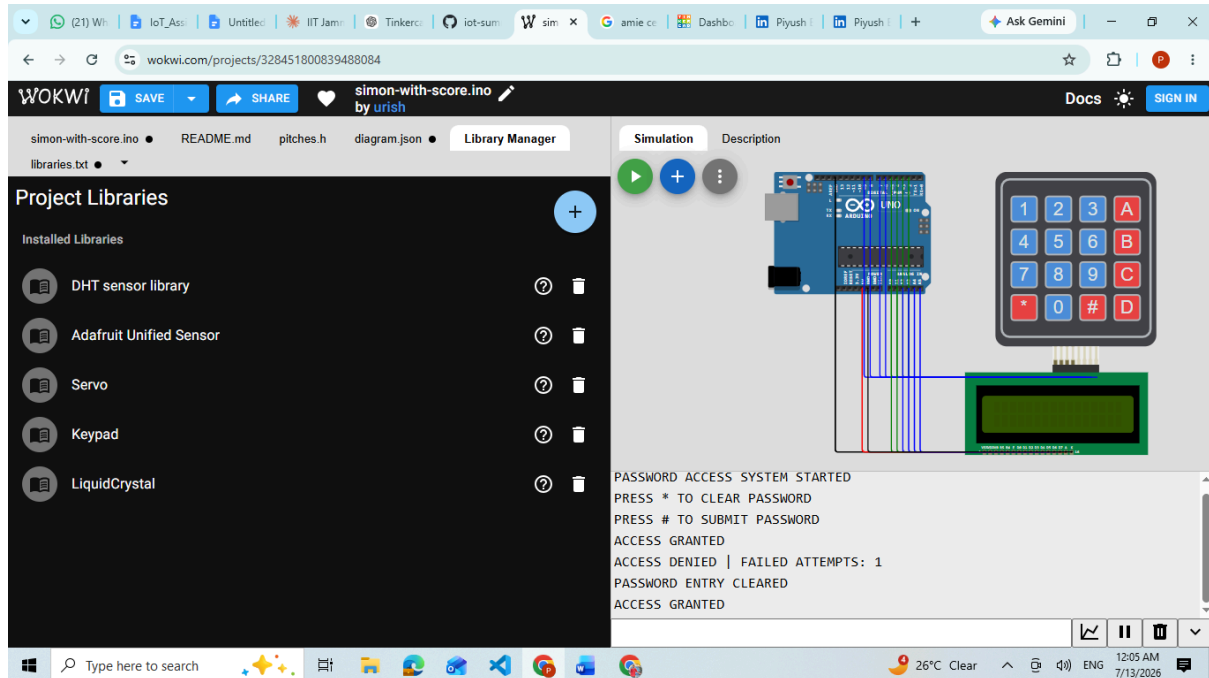
["lcd:RS", "uno:A0", "green", []],
["lcd:E", "uno:A1", "green", []],

["lcd:D4", "uno:A2", "blue", []],
["lcd:D5", "uno:A3", "blue", []],
["lcd:D6", "uno:A4", "blue", []],
["lcd:D7", "uno:A5", "blue", []],

["lcd:A", "uno:5V", "red", []],
["lcd:K", "uno:GND.2", "black", []]
],
"dependencies": {}
}

```

Circuit Diagram :-



Q29. What is sensor calibration and why is it important in IoT systems? How would you calibrate an analog soil moisture sensor to give accurate percentage readings? Describe the two-point calibration method. (2 marks)

Sensor calibration is the process of comparing sensor readings with known reference values to improve measurement accuracy and reliability. Calibration is important because sensors may produce errors due to manufacturing differences, environmental conditions, electrical noise, aging, and installation conditions.

In IoT systems, inaccurate sensor readings can lead to incorrect decisions. For example, an uncalibrated soil moisture sensor may unnecessarily turn ON an irrigation pump even when sufficient moisture is present.

TWO-POINT CALIBRATION

Two-point calibration uses two known reference conditions to convert raw sensor readings into meaningful values.

For a soil moisture sensor, the two calibration points are:

1. Dry Reference Value – represents approximately 0% moisture.
2. Wet Reference Value – represents approximately 100% moisture.

Suppose the experimentally measured values are:

Dry Value = 850

Wet Value = 350

The soil moisture percentage is calculated using:

Moisture Percentage = $((\text{Dry Value} - \text{Current Reading}) / (\text{Dry Value} - \text{Wet Value})) \times 100$

Example:

Current Reading = 600

Moisture Percentage = $((850 - 600) / (850 - 350)) \times 100$

Moisture Percentage = 50%

CALIBRATION PROCEDURE

First, place the sensor in the selected dry reference condition and record multiple readings. Calculate the average value and store it as the Dry Value.

Next, place the sensor in fully wet or water-saturated soil. Record multiple readings, calculate the average, and store it as the Wet Value.

The current sensor reading is then mapped between the dry and wet reference values to calculate the soil moisture percentage.

CONCLUSION

Sensor calibration improves the accuracy and reliability of measurements. Two-point calibration is a simple method for converting raw soil moisture sensor readings into approximate moisture percentages for smart irrigation and agricultural IoT applications.

Q30. Explain I2C protocol with respect to IoT sensor communication. What are SDA and SCL lines? How does I2C addressing work? Name 3 common IoT sensors that use I2C. (2 marks)

I2C stands for Inter-Integrated Circuit. It is a synchronous serial communication protocol widely used to connect microcontrollers with sensors, displays, memory devices, and other peripherals.

I2C uses only two communication lines:

1. SDA – Serial Data Line.
2. SCL – Serial Clock Line.

SDA transfers data between the controller and peripheral devices, while SCL carries the clock signal used to synchronize communication.

On Arduino UNO:

SDA = A4

SCL = A5

DEVICE ADDRESSING

Each I2C peripheral has an address. The controller sends the address of the required device before starting communication.

Because of device addressing, multiple I2C devices can share the same SDA and SCL lines.

For example:

OLED Display = 0x3C

BME280 Sensor = 0x76

MPU6050 Sensor = 0x68

BASIC I2C COMMUNICATION PROCESS

1. Controller generates a START condition.
2. Controller sends the device address.
3. Controller specifies Read or Write operation.
4. Peripheral sends an acknowledgement.

5. Data is transferred.
6. Controller generates a STOP condition.

ADVANTAGES OF I2C

I2C requires only two communication wires.

Multiple devices can share the same communication bus.

It reduces the number of microcontroller pins required.

It is widely supported by IoT sensors and microcontrollers.

LIMITATIONS OF I2C

I2C is generally slower than SPI.

It is mainly suitable for short-distance communication.

Address conflicts may occur when devices have the same fixed address.

Pull-up resistors are required on SDA and SCL lines.

EXAMPLES OF I2C SENSORS

1. BME280 – Measures temperature, humidity, and atmospheric pressure.
2. MPU6050 – Measures acceleration and angular velocity.
3. BH1750 – Measures light intensity in lux.

CONCLUSION

I2C is a two-wire serial communication protocol widely used in IoT and embedded systems. SDA transfers data, SCL provides the clock signal, and device addressing allows multiple sensors to communicate using the same two wires.

Q31. Bluetooth LED Controller (HC-05 + Arduino)

Main Code :-

/*

Q31 - Bluetooth LED Controller

Physical Hardware:

Arduino UNO + HC-05 Bluetooth Module

Wokwi Simulation:

Commands are entered through Serial Monitor.

Commands:

R -> Red LED ON

G -> Green LED ON

B -> Blue LED ON

X -> All LEDs OFF

F -> Flash all LEDs 5 times

*/

```
#include <SoftwareSerial.h>
```

```
const int HC05_RX = 10;
```

```
const int HC05_TX = 11;
```

```
const int RED_LED = 5;
```

```
const int GREEN_LED = 6;
```

```
const int BLUE_LED = 7;
```

```
SoftwareSerial bluetooth(HC05_RX, HC05_TX);
```

```
void allLEDsOff() {
```

```
    digitalWrite(RED_LED, LOW);
```

```
    digitalWrite(GREEN_LED, LOW);
```

```
    digitalWrite(BLUE_LED, LOW);
```

```
}
```

```
void flashAllLEDs() {
```

```
    Serial.println("FLASHING ALL LEDs 5 TIMES");
```

```
    for (int i = 0; i < 5; i++) {
```

```
        digitalWrite(RED_LED, HIGH);
```

```
        digitalWrite(GREEN_LED, HIGH);
```

```
        digitalWrite(BLUE_LED, HIGH);
```

```
        delay(500);
```

```
        allLEDsOff();
```

```
        delay(500);
```

```
    }
```

```

Serial.println("FLASH COMPLETED");
}

void processCommand(char command) {
  if (command >= 'a' && command <= 'z') {
    command = command - 32;
  }

  switch (command) {

    case 'R':
      allLEDsOff();
      digitalWrite(RED_LED, HIGH);
      Serial.println("RED LED ON");
      break;

    case 'G':
      allLEDsOff();
      digitalWrite(GREEN_LED, HIGH);
      Serial.println("GREEN LED ON");
      break;

    case 'B':
      allLEDsOff();
      digitalWrite(BLUE_LED, HIGH);
      Serial.println("BLUE LED ON");
      break;

    case 'X':
      allLEDsOff();
      Serial.println("ALL LEDs OFF");
      break;

    case 'F':
      flashAllLEDs();
      break;

    case '\n':
    case '\r':
      break;

    default:
      Serial.println("INVALID COMMAND");
      break;
  }
}

```

```

void setup() {
  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(BLUE_LED, OUTPUT);

  allLEDsOff();

  Serial.begin(9600);

  bluetooth.begin(9600);

  Serial.println("BLUETOOTH LED CONTROLLER STARTED");
  Serial.println("COMMANDS: R, G, B, X, F");
}

void loop() {
  // Physical HC-05 Bluetooth input

  if (bluetooth.available()) {
    char command = bluetooth.read();

    processCommand(command);
  }

  // Wokwi Serial Monitor input

  if (Serial.available()) {
    char command = Serial.read();

    processCommand(command);
  }
}

```

Diagram Code :-

```

{
  "version": 1,
  "author": "Piyush Bhamare",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-arduino-uno",
      "id": "uno",
      "top": 20,
      "left": 20,
      "attrs": {}
    },
    {
      "type": "wokwi-led",

```

```
"id": "redLed",
"top": 20,
"left": 400,
"attrs": {
  "color": "red",
  "label": "RED"
}
},
{
  "type": "wokwi-resistor",
  "id": "r1",
  "top": 100,
  "left": 400,
  "attrs": {
    "value": "220"
  }
},
{
  "type": "wokwi-led",
  "id": "greenLed",
  "top": 180,
  "left": 400,
  "attrs": {
    "color": "green",
    "label": "GREEN"
  }
},
{
  "type": "wokwi-resistor",
  "id": "r2",
  "top": 260,
  "left": 400,
  "attrs": {
    "value": "220"
  }
},
{
  "type": "wokwi-led",
  "id": "blueLed",
  "top": 340,
  "left": 400,
  "attrs": {
    "color": "blue",
    "label": "BLUE"
  }
},
{
  "type": "wokwi-resistor",
```

```

    "id": "r3",
    "top": 420,
    "left": 400,
    "attrs": {
      "value": "220"
    }
  }
],
"connections": [
  ["uno:5", "redLed:A", "red", []],
  ["redLed:C", "r1:1", "black", []],
  ["r1:2", "uno:GND.1", "black", []],

  ["uno:6", "greenLed:A", "green", []],
  ["greenLed:C", "r2:1", "black", []],
  ["r2:2", "uno:GND.1", "black", []],

  ["uno:7", "blueLed:A", "blue", []],
  ["blueLed:C", "r3:1", "black", []],
  ["r3:2", "uno:GND.2", "black", []]
],
"dependencies": {}
}

```

Circuit Diagram :-

The screenshot displays the Wokwi web IDE interface. On the left, the code editor shows the following C++ code:

```

97 void setup() {
103   Serial.begin(9600);
104
105   bluetooth.begin(9600);
106
107   Serial.println("BLUETOOTH LED CONTROLLER STARTED");
108   Serial.println("COMMANDS: R, G, B, X, F");
109 }
110
111
112 void loop() {
113   // Physical HC-05 Bluetooth input
114
115   if (bluetooth.available()) {
116     char command = bluetooth.read();
117     processCommand(command);
118   }
119
120   // Wokwi Serial Monitor input
121
122   if (Serial.available()) {
123     char command = Serial.read();
124     processCommand(command);
125   }
126 }
127
128

```

On the right, the simulation window shows a circuit diagram of an Arduino Uno connected to three LEDs (red, green, blue) through resistors. Below the diagram, the serial monitor output is visible:

```

BLUETOOTH LED CONTROLLER STARTED
COMMANDS: R, G, B, X, F
INVALID COMMAND

```

The interface also includes a top navigation bar with options like 'SAVE', 'SHARE', and 'SIGN UP', and a bottom status bar showing system information like temperature and time.

Q32. Bluetooth Sensor Dashboard (ESP32 BLE)

Main Code :-

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

#include <DHT.h>

#define DHT_PIN 15
#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);

#define SERVICE_UUID
"12345678-1234-1234-1234-123456789000"

#define TEMPERATURE_UUID
"12345678-1234-1234-1234-123456789001"

#define HUMIDITY_UUID
"12345678-1234-1234-1234-123456789002"

#define TIMESTAMP_UUID
"12345678-1234-1234-1234-123456789003"

BLECharacteristic *temperatureCharacteristic;

BLECharacteristic *humidityCharacteristic;

BLECharacteristic *timestampCharacteristic;

bool deviceConnected = false;

class ServerCallbacks : public BLEServerCallbacks {

    void onConnect(BLEServer *server) {
        deviceConnected = true;

        Serial.println("BLE CLIENT CONNECTED");
    }

    void onDisconnect(BLEServer *server) {
        deviceConnected = false;

        Serial.println("BLE CLIENT DISCONNECTED");

        BLEDevice::startAdvertising();
    }
}
```

```

};

void setup() {
  Serial.begin(115200);

  dht.begin();

  BLEDevice::init("IITJammu-BLE-Sensor");

  BLEServer *server =
    BLEDevice::createServer();

  server->setCallbacks(
    new ServerCallbacks()
  );

  BLEService *service =
    server->createService(SERVICE_UUID);

  temperatureCharacteristic =
    service->createCharacteristic(
      TEMPERATURE_UUID,
      BLECharacteristic::PROPERTY_READ |
      BLECharacteristic::PROPERTY_NOTIFY
    );

  humidityCharacteristic =
    service->createCharacteristic(
      HUMIDITY_UUID,
      BLECharacteristic::PROPERTY_READ |
      BLECharacteristic::PROPERTY_NOTIFY
    );

  timestampCharacteristic =
    service->createCharacteristic(
      TIMESTAMP_UUID,
      BLECharacteristic::PROPERTY_READ |
      BLECharacteristic::PROPERTY_NOTIFY
    );

  temperatureCharacteristic->addDescriptor(
    new BLE2902()
  );

  humidityCharacteristic->addDescriptor(
    new BLE2902()
  );

```



```

timestampCharacteristic->addDescriptor(
    new BLE2902()
);

service->start();

BLEAdvertising *advertising =
    BLEDevice::getAdvertising();

advertising->addServiceUUID(SERVICE_UUID);

advertising->setScanResponse(true);

BLEDevice::startAdvertising();

Serial.println("ESP32 BLE SENSOR DASHBOARD STARTED");

Serial.println("WAITING FOR BLE CLIENT...");
}

void loop() {
    float temperature =
        dht.readTemperature();

    float humidity =
        dht.readHumidity();

    unsigned long timestamp =
        millis();

    if (isnan(temperature) ||
        isnan(humidity)) {

        Serial.println("FAILED TO READ DHT11");

        delay(3000);

        return;
    }

    String temperatureData =
        String(temperature, 1) + " C";

    String humidityData =
        String(humidity, 1) + " %";

    String timestampData =
        String(timestamp);

```

```

temperatureCharacteristic->setValue(
    temperatureData.c_str()
);

humidityCharacteristic->setValue(
    humidityData.c_str()
);

timestampCharacteristic->setValue(
    timestampData.c_str()
);

if (deviceConnected) {
    temperatureCharacteristic->notify();

    humidityCharacteristic->notify();

    timestampCharacteristic->notify();
}

Serial.print("TEMPERATURE: ");
Serial.print(temperatureData);

Serial.print(" | HUMIDITY: ");
Serial.print(humidityData);

Serial.print(" | TIMESTAMP: ");
Serial.println(timestampData);

delay(3000);
}

```

Diagram Code :-

```

{
  "version": 1,
  "author": "Piyush Bhamare",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-devkit-c-v4",
      "id": "esp32",
      "top": 20,
      "left": 20,
      "attrs": {}
    },
    {
      "type": "wokwi-dht22",

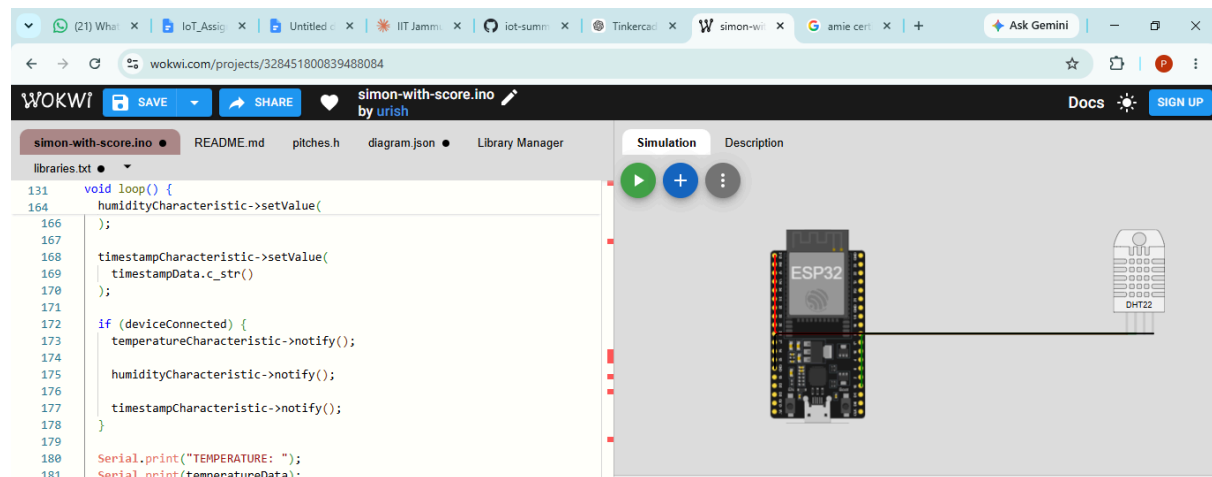
```

```

    "id": "dht",
    "top": 20,
    "left": 400,
    "attrs": {
      "temperature": "28",
      "humidity": "60"
    }
  },
  "connections": [
    ["dht:VCC", "esp32:3V3", "red", []],
    ["dht:SDA", "esp32:15", "green", []],
    ["dht:GND", "esp32:GND.1", "black", []]
  ],
  "dependencies": {}
}

```

Circuit Diagram :-



Q33. Wi-Fi Temperature Monitor (ESP8266/ESP32)

Main Code :-

```

#include <WiFi.h>
#include <WebServer.h>
#include <DHT.h>

const char *ssid = "Wokwi-GUEST";
const char *password = "";

#define DHT_PIN 15
#define DHT_TYPE DHT22

#define LED_PIN 2

DHT dht(DHT_PIN, DHT_TYPE);

```

```

WebServer server(80);

bool ledState = false;

String getUptime() {
    unsigned long totalSeconds =
        millis() / 1000;

    unsigned long hours =
        totalSeconds / 3600;

    unsigned long minutes =
        (totalSeconds % 3600) / 60;

    unsigned long seconds =
        totalSeconds % 60;

    String uptime = "";

    uptime += String(hours);
    uptime += "h ";

    uptime += String(minutes);
    uptime += "m ";

    uptime += String(seconds);
    uptime += "s";

    return uptime;
}

String createWebPage() {
    float temperature =
        dht.readTemperature();

    float humidity =
        dht.readHumidity();

    String page = "";

    page += "<!DOCTYPE html>";

    page += "<html>";

    page += "<head>";

    page += "<meta name='viewport' ";

```

```
page += "content='width=device-width, initial-scale=1'>";
```

```
page += "<meta http-equiv='refresh' content='10'>";
```

```
page += "<title>IIT Jammu Temperature Monitor</title>";
```

```
page += "</head>";
```

```
page += "<body>";
```

```
page += "<h1>Wi-Fi Temperature Monitor</h1>";
```

```
page += "<h2>Temperature: ";
```

```
if (isnan(temperature)) {  
    page += "Sensor Error";  
}  
else {  
    page += String(temperature, 1);  
    page += " &deg;C";  
}
```

```
page += "</h2>";
```

```
page += "<h2>Humidity: ";
```

```
if (isnan(humidity)) {  
    page += "Sensor Error";  
}  
else {  
    page += String(humidity, 1);  
    page += " %";  
}
```

```
page += "</h2>";
```

```
page += "<h2>LED State: ";
```

```
if (ledState) {  
    page += "ON";  
}  
else {  
    page += "OFF";  
}
```

```
page += "</h2>";
```

```
page += "<a href='/toggle'>";
```

```

page += "<button style='font-size:24px;padding:15px;'>";

page += "TOGGLE LED";

page += "</button>";

page += "</a>";

page += "<h3>System Uptime: ";

page += getUptime();

page += "</h3>";

page += "<p>Page Auto Refreshes Every 10 Seconds</p>";

page += "</body>";

page += "</html>";

return page;
}

void handleRoot() {
  server.send(
    200,
    "text/html",
    createWebPage()
  );
}

void handleToggle() {
  ledState = !ledState;

  digitalWrite(
    LED_PIN,
    ledState ? HIGH : LOW
  );

  Serial.print("LED STATE CHANGED: ");

  if (ledState) {
    Serial.println("ON");
  }
  else {
    Serial.println("OFF");
  }
}

```

```

server.sendHeader(
  "Location",
  "/"
);

server.send(
  303,
  "text/plain",
  ""
);
}

void setup() {
  Serial.begin(115200);

  pinMode(LED_PIN, OUTPUT);

  digitalWrite(LED_PIN, LOW);

  dht.begin();

  Serial.println("CONNECTING TO WIFI...");

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);

    Serial.print(".");
  }

  Serial.println();

  Serial.println("WIFI CONNECTED");

  Serial.print("LOCAL IP ADDRESS: ");

  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);

  server.on("/toggle", handleToggle);

  server.begin();

  Serial.println("WEB SERVER STARTED");
}

```

```
void loop() {  
  server.handleClient();  
}
```

Diagram Code :-

```
{  
  "version": 1,  
  "author": "Piyush Bhamare",  
  "editor": "wokwi",  
  "parts": [  
    {  
      "type": "board-esp32-devkit-c-v4",  
      "id": "esp32",  
      "top": 20,  
      "left": 20,  
      "attrs": {}  
    },  
    {  
      "type": "wokwi-dht22",  
      "id": "dht",  
      "top": 20,  
      "left": 400,  
      "attrs": {  
        "temperature": "28",  
        "humidity": "60"  
      }  
    },  
    {  
      "type": "wokwi-led",  
      "id": "led",  
      "top": 250,  
      "left": 400,  
      "attrs": {  
        "color": "green"  
      }  
    },  
    {  
      "type": "wokwi-resistor",  
      "id": "resistor",  
      "top": 340,  
      "left": 400,  
      "attrs": {  
        "value": "220"  
      }  
    }  
  ],  
}
```



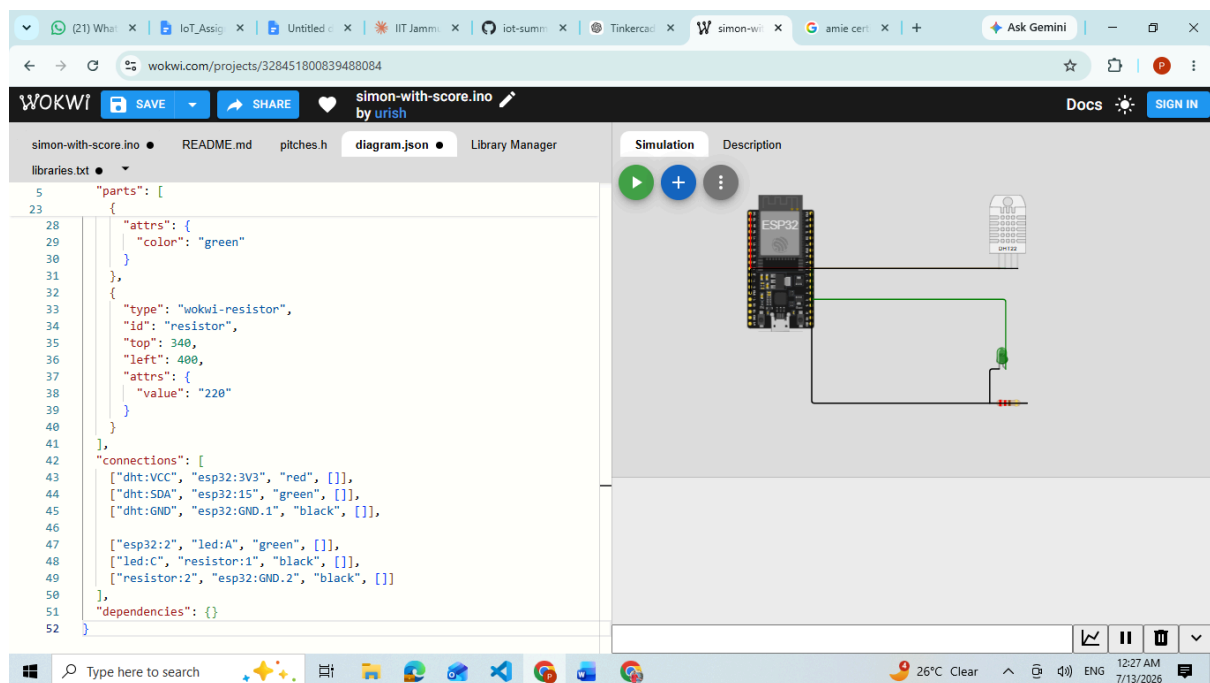
```

"connections": [
  ["dht:VCC", "esp32:3V3", "red", []],
  ["dht:SDA", "esp32:15", "green", []],
  ["dht:GND", "esp32:GND.1", "black", []],

  ["esp32:2", "led:A", "green", []],
  ["led:C", "resistor:1", "black", []],
  ["resistor:2", "esp32:GND.2", "black", []]
],
"dependencies": {}
}

```

Circuit Diagram :-



Q34. MQTT Sensor Publisher

Main Code :-

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

const char *ssid = "Wokwi-GUEST";
const char *password = "";

const char *mqttBroker =
    "broker.hivemq.com";

const int mqttPort = 1883;

const char *temperatureTopic =
    "iitjammu/summer26/piyushbhamare/temperature";

const char *humidityTopic =
    "iitjammu/summer26/piyushbhamare/humidity";

const char *ledControlTopic =
    "iitjammu/summer26/piyushbhamare/led_control";

#define DHT_PIN 15
#define DHT_TYPE DHT22

#define LED_PIN 2

DHT dht(DHT_PIN, DHT_TYPE);

WiFiClient wifiClient;

PubSubClient mqttClient(wifiClient);

unsigned long previousPublishTime = 0;

const unsigned long PUBLISH_INTERVAL = 5000;

void mqttCallback(
    char *topic,
    byte *payload,
    unsigned int length
) {
    String message = "";

    for (unsigned int i = 0; i < length; i++) {
        message += (char)payload[i];
    }

    message.trim();
```

```
Serial.print("MESSAGE RECEIVED");

Serial.print(" | TOPIC: ");

Serial.print(topic);

Serial.print(" | MESSAGE: ");

Serial.println(message);

if (String(topic) == ledControlTopic) {

    if (message == "ON" ||
        message == "1") {

        digitalWrite(LED_PIN, HIGH);

        Serial.println("LED TURNED ON");
    }

    else if (message == "OFF" ||
             message == "0") {

        digitalWrite(LED_PIN, LOW);

        Serial.println("LED TURNED OFF");
    }
}

void connectWiFi() {
    Serial.print("CONNECTING TO WIFI");

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);

        Serial.print(".");
    }

    Serial.println();

    Serial.println("WIFI CONNECTED");

    Serial.print("IP ADDRESS: ");
```

```

    Serial.println(WiFi.localIP());
}

void connectMQTT() {
    while (!mqttClient.connected()) {

        Serial.print("CONNECTING TO MQTT BROKER...");

        String clientId =
            "IITJammuESP32-";

        clientId +=
            String(
                (uint32_t)ESP.getEfuseMac(),
                HEX
            );

        if (mqttClient.connect(clientId.c_str())) {

            Serial.println("CONNECTED");

            mqttClient.subscribe(ledControlTopic);

            Serial.print("SUBSCRIBED TO: ");

            Serial.println(ledControlTopic);
        }

        else {
            Serial.print("FAILED, STATE = ");

            Serial.println(mqttClient.state());

            delay(5000);
        }
    }
}

void publishSensorData() {
    float temperature =
        dht.readTemperature();

    float humidity =
        dht.readHumidity();

    if (isnan(temperature) ||
        isnan(humidity)) {

```

```

    Serial.println("FAILED TO READ DHT SENSOR");

    return;
}

unsigned long timestamp =
    millis() / 1000;

String temperatureJSON = "";

temperatureJSON += "{\"value\":";

temperatureJSON += String(temperature, 1);

temperatureJSON += ", \"unit\": \"C\", \"ts\":";

temperatureJSON += String(timestamp);

temperatureJSON += "}";

String humidityJSON = "";

humidityJSON += "{\"value\":";

humidityJSON += String(humidity, 1);

humidityJSON += ", \"unit\": \"%\", \"ts\":";

humidityJSON += String(timestamp);

humidityJSON += "}";

mqttClient.publish(
    temperatureTopic,
    temperatureJSON.c_str()
);

mqttClient.publish(
    humidityTopic,
    humidityJSON.c_str()
);

Serial.print("PUBLISHED TEMPERATURE: ");

Serial.println(temperatureJSON);

Serial.print("PUBLISHED HUMIDITY: ");

```

```

    Serial.println(humidityJSON);
}

void setup() {
    Serial.begin(115200);

    pinMode(LED_PIN, OUTPUT);

    digitalWrite(LED_PIN, LOW);

    dht.begin();

    connectWiFi();

    mqttClient.setServer(
        mqttBroker,
        mqttPort
    );

    mqttClient.setCallback(mqttCallback);

    connectMQTT();

    previousPublishTime = millis() - PUBLISH_INTERVAL;
}

void loop() {
    if (WiFi.status() != WL_CONNECTED) {
        connectWiFi();
    }

    if (!mqttClient.connected()) {
        connectMQTT();
    }

    mqttClient.loop();

    unsigned long currentTime = millis();

    if (currentTime - previousPublishTime >=
        PUBLISH_INTERVAL) {

        previousPublishTime = currentTime;

        publishSensorData();
    }
}

```

Diagram Code :-

```
{
  "version": 1,
  "author": "Piyush Bhamare",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-devkit-c-v4",
      "id": "esp32",
      "top": 20,
      "left": 20,
      "attrs": {}
    },
    {
      "type": "wokwi-dht22",
      "id": "dht",
      "top": 20,
      "left": 400,
      "attrs": {
        "temperature": "28.5",
        "humidity": "60"
      }
    },
    {
      "type": "wokwi-led",
      "id": "led",
      "top": 250,
      "left": 400,
      "attrs": {
        "color": "green",
        "label": "MQTT LED"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "resistor",
      "top": 340,
      "left": 400,
      "attrs": {
        "value": "220"
      }
    }
  ],
  "connections": [
    ["dht:VCC", "esp32:3V3", "red", []],
    ["dht:SDA", "esp32:15", "green", []],
    ["dht:GND", "esp32:GND.1", "black", []],
```

```

["esp32:2", "led:A", "green", []],
["led:C", "resistor:1", "black", []],
["resistor:2", "esp32:GND.2", "black", []]
],
"dependencies": {}
}

```

Circuit Diagram :-

The screenshot shows the Wokwi IoT simulator interface. The left pane displays the code for an MQTT Sensor Publisher using an ESP32 board. The right pane shows a simulation of the ESP32 connected to an LED and a resistor. The bottom status bar indicates a temperature of 26°C and the date 7/13/2026.

Code (simon-with-score.ino):

```

1 //
2 Q34 - MQTT Sensor Publisher
3
4 Board:
5 ESP32
6
7 Broker:
8 broker.hivemq.com
9
10 Publish Interval:
11 5 Seconds
12
13 Published Data:
14 Temperature JSON
15 Humidity JSON
16
17 Subscriber:
18 LED Control Topic
19 */
20
21 #include <WiFi.h>
22 #include <PubSubClient.h>
23 #include <DHT.h>
24
25 const char *ssid = "Wokwi-GUEST";
26 const char *password = "";
27
28 const char *mqttBroker =

```

Simulation Details:

- Board: ESP32
- Broker: broker.hivemq.com
- Publish Interval: 5 Seconds
- Published Data: Temperature JSON, Humidity JSON
- Subscriber: LED Control Topic
- Simulation Time: 00:13.925
- Battery Level: 99%